

第8章 线性规划 (Linear Programming)

学习要点

- 理解线性规划问题建模
- 掌握解线性规划问题的**单纯形算法**
- 仓库租赁问题
- 最大网络流问题、增广路算法

线性规划(Linear Programming, LP)

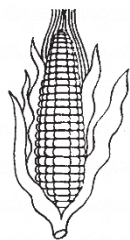
线性规划是运筹学的一个重要分支，广泛应用于军事作战、经济分析、经营管理和工程技术等方面，为合理地利用有限的人力、物力、财力等资源作出的最优决策，提供科学的依据。

研究线性约束条件下线性目标函数的极值问题的数学理论和方法，是一系列竞争活动中优化稀缺资源分配问题的解决模型，包括：最短路径、最大流量、匹配、分配、零和博弈 ...

$$\begin{array}{ll} \text{maximize} & 13A + 23B \\ \text{subject to the constraints} & \left\{ \begin{array}{ll} 5A + 15B & \leq 480 \\ 4A + 4B & \leq 160 \\ 35A + 20B & \leq 1190 \\ A, B & \geq 0 \end{array} \right. \end{array}$$

示例：啤酒厂问题

小型啤酒厂生产麦芽酒和啤酒，生产资源受限。麦芽酒和啤酒的食谱需要不同比例的资源。



玉米 (480磅)

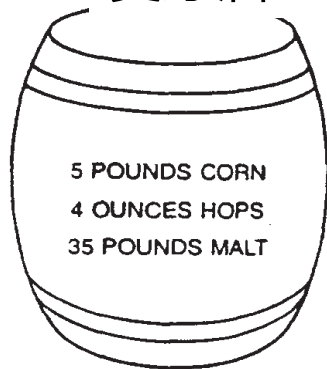


啤酒花 (160盎司)



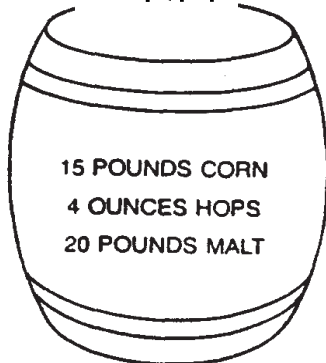
麦芽 (1190磅)

麦芽酒



每桶利润13美元

啤酒



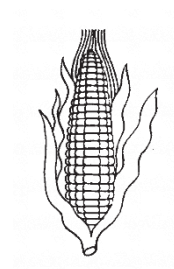
每桶利润23美元

示例：啤酒厂问题

啤酒厂问题：选择产品组合以实现利润最大化。

34桶×35磅麦芽=1190磅[可用麦芽量]

麦芽酒	啤酒	玉米	啤酒花	麦芽	利润
34	0	179	136	1190	\$442
0	32	480	128	640	\$736
19.5	20.5	405	160	1092.5	\$725
12	28	480	160	980	\$800
?	?				> \$800 ?



corn (480 lbs)

玉米



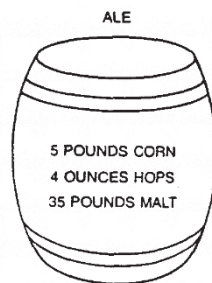
hops (160 oz)

啤酒花



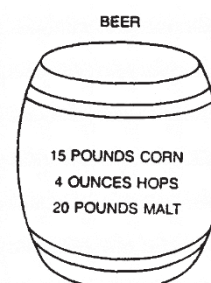
malt (1190 lbs)

麦芽



\$13 profit per barrel

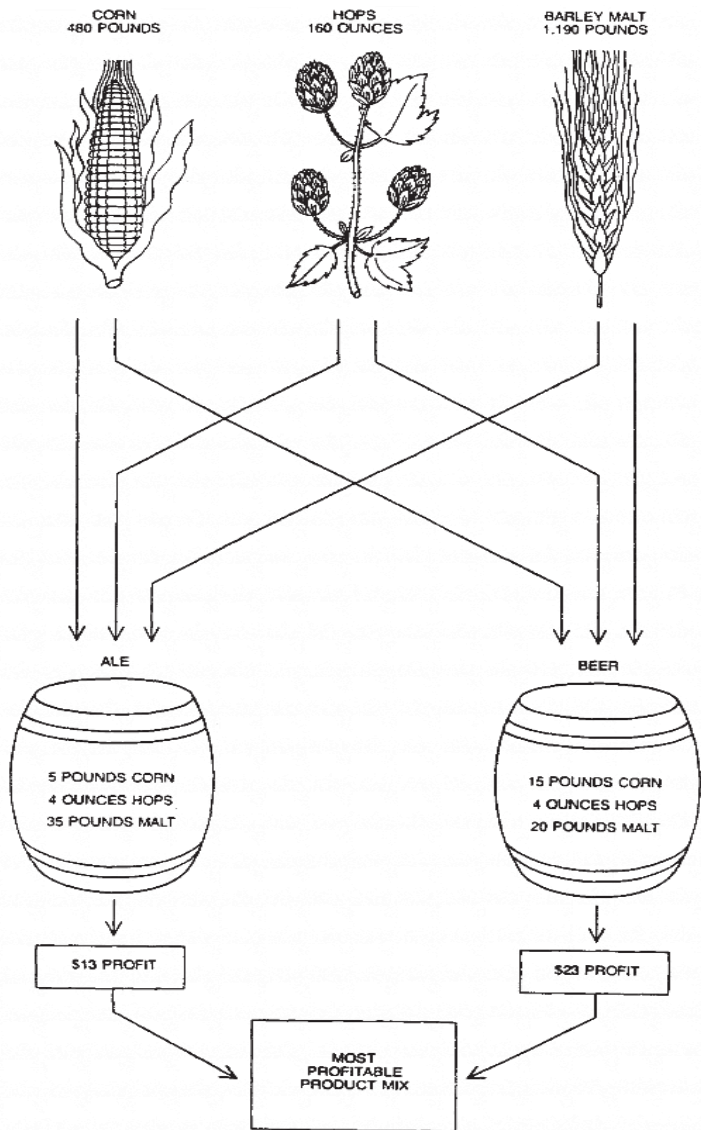
麦芽酒



\$23 profit per barrel

啤酒

示例：啤酒厂问题



设A为麦芽酒的桶数；B为啤酒桶数

线性规划公式：

$$\begin{aligned} &\text{maximize} && 13A + 23B \\ &\text{s.t.} && 5A + 15B \leq 480 \\ &&& 4A + 4B \leq 160 \\ &&& 35A + 20B \leq 1190 \\ &&& A, B \geq 0 \end{aligned}$$

8.1 线性规划问题和单纯形算法

■ 线性规划问题及其形式化表示

总共 n 个变量, m 个约束条件

$$\max \quad \sum_{j=1}^n c_j x_j \quad (8.1)$$

s.t.

$$\sum_{t=1}^n a_{it} x_t \leq b_i \quad i = 1, 2, \dots, m_1 \quad (8.2)$$

$$\sum_{t=1}^n a_{jt} x_t = b_j \quad j = m_1 + 1, \dots, m_1 + m_2 \quad (8.3)$$

$$\sum_{t=1}^n a_{kt} x_t \geq b_k \quad k = m_1 + m_2 + 1, \dots, m_1 + m_2 + m_3 \quad (8.4)$$

变量非负约束

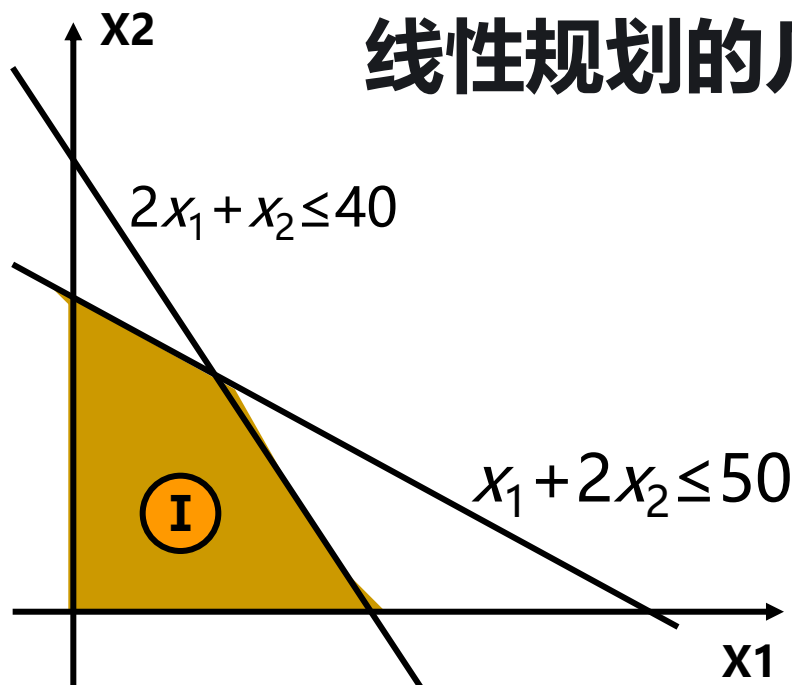
$$x_t \geq 0 \quad t = 1, 2, \dots, n \quad (8.5)$$

基本概念：可行解、最优解、最优值

线性规划解和若干概念

$$\begin{array}{ll}\max & z = 5x_1 + 3x_2 \\ \text{s.t.} & 2x_1 + x_2 \leq 40 \\ & x_1 + 2x_2 \leq 50 \\ & x_1, x_2 \geq 0\end{array}$$

线性规划的几何含义



可行解

满足所有约束条件的解。在二维中，位于所有约束半平面交集内的点（如图中阴影区域内的任意点）

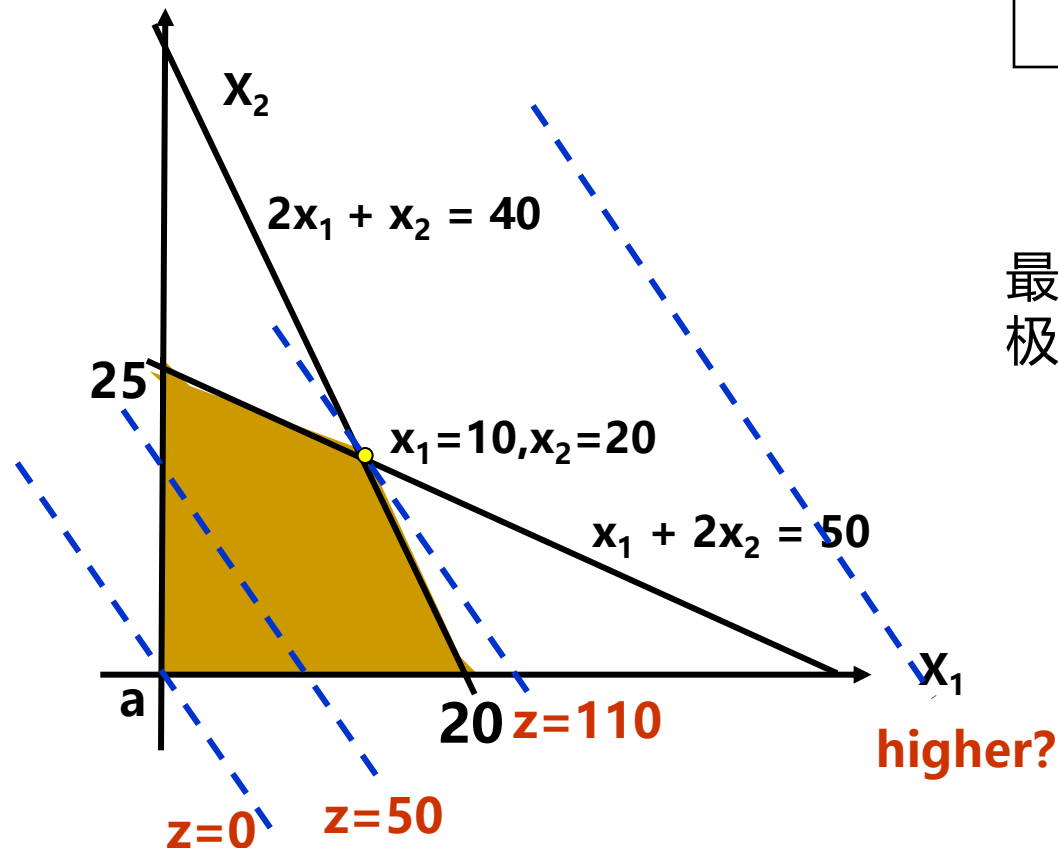
可行区域 I

可行域内部的点有可能是最优解？

线性规划模型

$$\begin{aligned} \max \quad & z = 5x_1 + 3x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \leq 40 \\ & x_1 + 2x_2 \leq 50 \\ & x_1, x_2 \geq 0 \end{aligned}$$

线性规划解的图示



最优解：使目标函数取得极值的可行解

最优解(10,20)

最优值110

求解的基本思想：沿着使得目标函数值增加的方向搜索最优解

线性规划解和若干概念

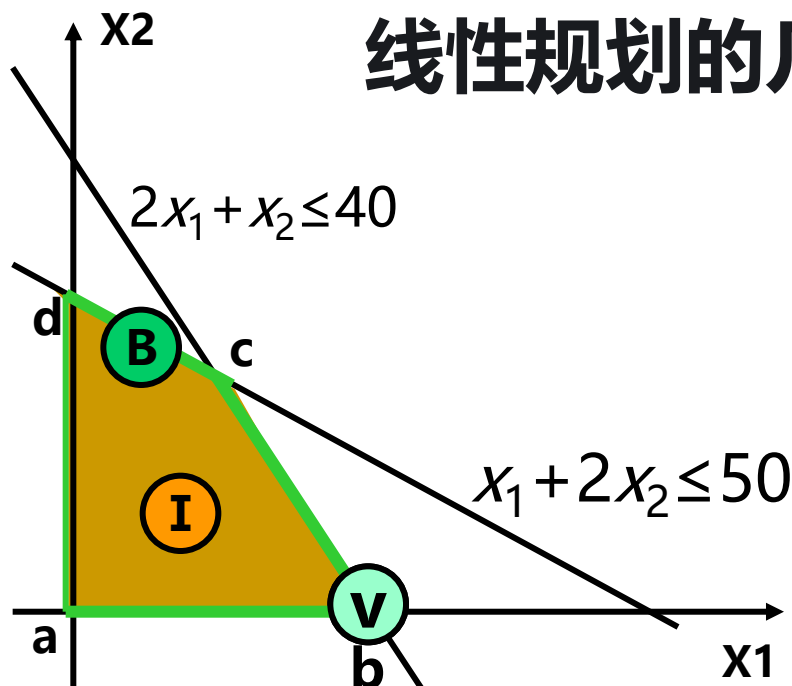
$$\max z = 5x_1 + 3x_2$$

$$\text{s.t. } 2x_1 + x_2 \leq 40$$

$$x_1 + 2x_2 \leq 50$$

$$x_1, x_2 \geq 0$$

线性规划的几何含义



可行解 满足所有约束条件的解。在二维中，位于所有约束半平面交集内的点（如图中阴影区域内的任意点）

可行域边界B 可行域边界的一部分，由两个基本可行解（顶点）确定。

基本可行解V 约束条件中某 n 个约束以等号满足的可行解(可行域极点)

8.1.2 线性规划基本定理

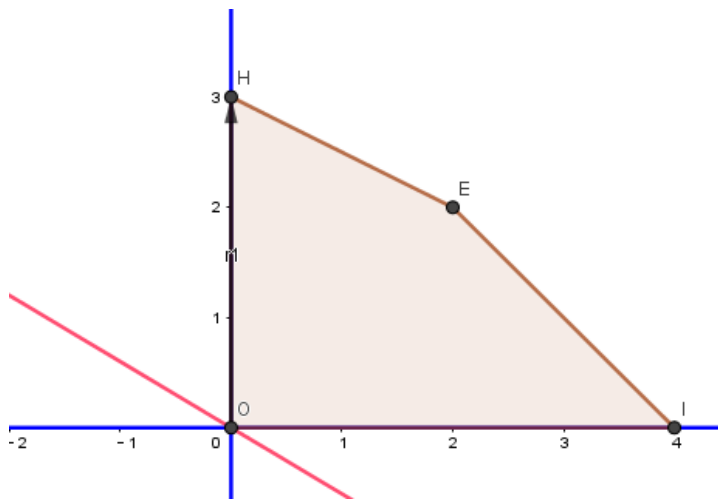
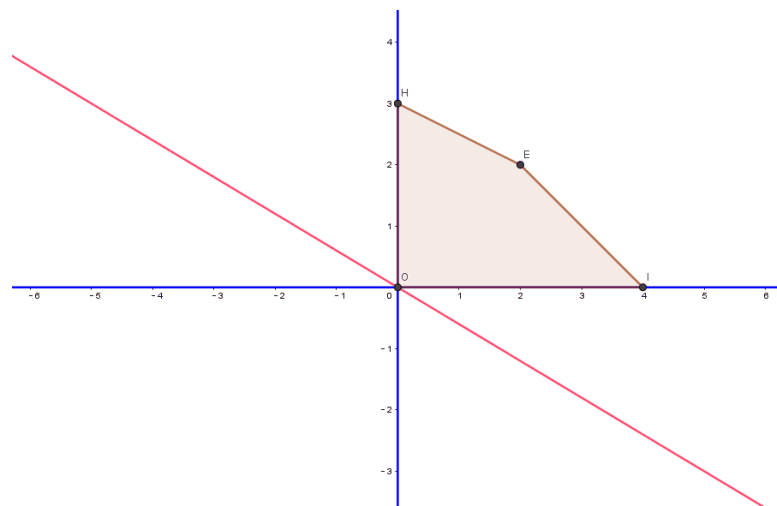
- **线性规划基本定理：**如果线性规划问题有最优解，则必有一基本可行最优解。
- 上述定理的重要意义在于：它把一个最优化问题转化为一个组合问题，即在(8.2) -(8.5)式的 $m+n$ 个约束条件中，确定最优解应满足其中哪 n 个约束条件的问题。
- 由此可知，只要对各种不同的组合进行测试，并比较每种情况下的目标函数值，直到找到最优解。

单纯形算法

- 1948年丹齐格(Dantzig) 提出了线性规划问题的**单纯形算法**。
- 单纯形算法的特点是：
 - (1) 只对约束条件的若干组合进行测试，测试的每一步都使目标函数的值增加；
 - (2) 一般经过不大于 m 或 n 次迭代就可求得最优解。

线性规划单纯形算法图示

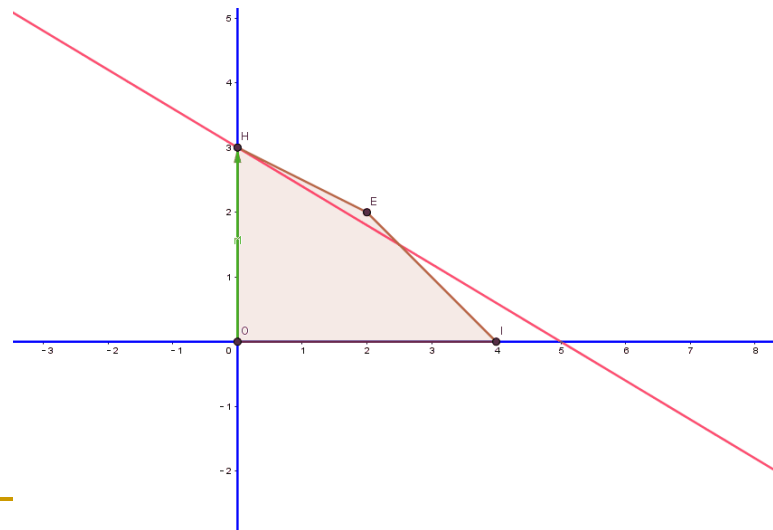
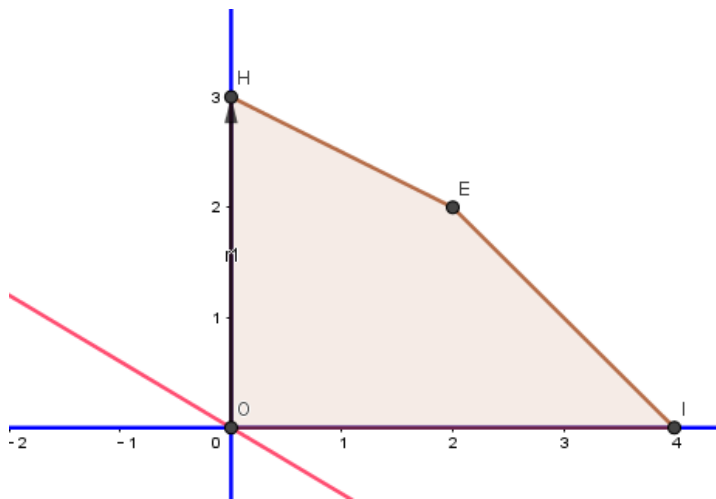
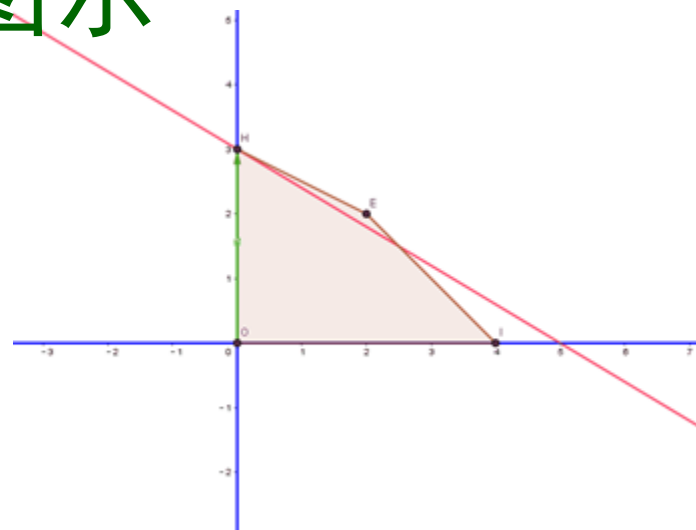
$$\begin{array}{ll}\max & 3x + 5y \\ \text{s.t.} & \begin{cases} x + y \leq 4 \\ x + 2y \leq 6 \\ x \geq 0, y \geq 0 \end{cases}\end{array}$$



求解的基本思想：沿着使得目标函数值增加的方向搜索最优解

线性规划单纯形算法图示

$$\begin{array}{ll}\max & 3x + 5y \\ \text{s.t.} & \begin{cases} x + y \leq 4 \\ x + 2y \leq 6 \\ x \geq 0, y \geq 0 \end{cases}\end{array}$$



求解的基本思想：沿着使得目标函数值增加的方向搜索最优解

8.1 线性规划问题和单纯形算法（回顾）

■ 线性规划问题及其形式化表示

总共 n 个变量, m 个约束条件

$$\max \quad \sum_{j=1}^n c_j x_j \quad (8.1)$$

s.t.
$$\sum_{t=1}^n a_{it} x_t \leq b_i \quad i = 1, 2, \dots, m_1 \quad (8.2)$$

$$\sum_{t=1}^n a_{jt} x_t = b_j \quad j = m_1 + 1, \dots, m_1 + m_2 \quad (8.3)$$

$$\sum_{t=1}^n a_{kt} x_t \geq b_k \quad k = m_1 + m_2 + 1, \dots, m_1 + m_2 + m_3 \quad (8.4)$$

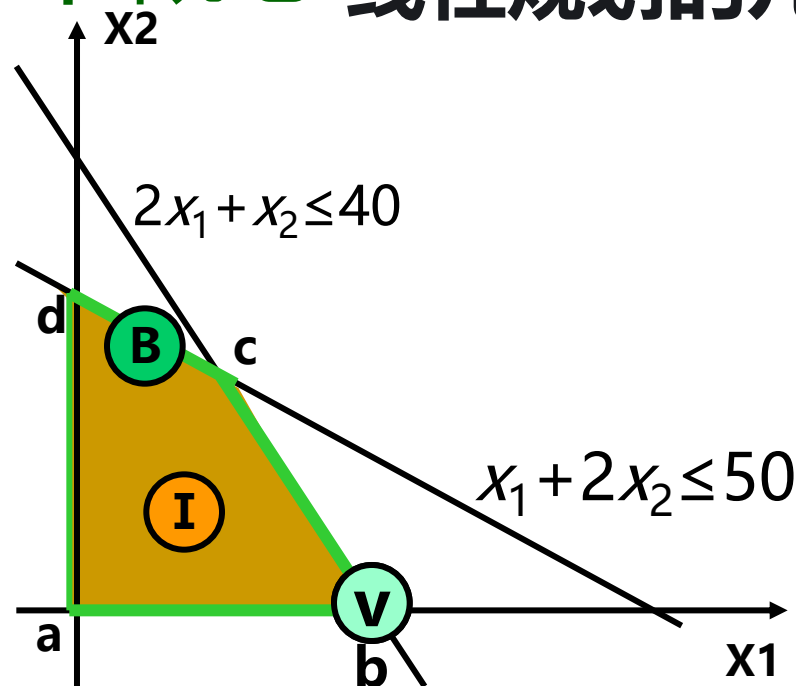
变量非负约束
$$x_t \geq 0 \quad t = 1, 2, \dots, n \quad (8.5)$$

基本概念：可行解、最优解、最优值

线性规划解和若干概念

线性规划的几何含义

$$\begin{aligned} \max \quad & z = 5x_1 + 3x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \leq 40 \\ & x_1 + 2x_2 \leq 50 \\ & x_1, x_2 \geq 0 \end{aligned}$$



可行解 满足所有约束条件的解（阴影区域内的任意点）

可行域I 是一个凸多面体

可行域边界B 可行域边界的一部分，由两个基本可行解（顶点）确定。

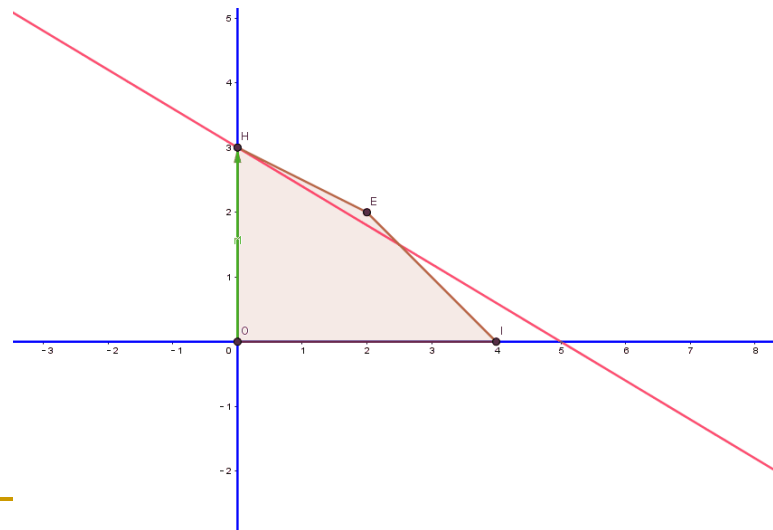
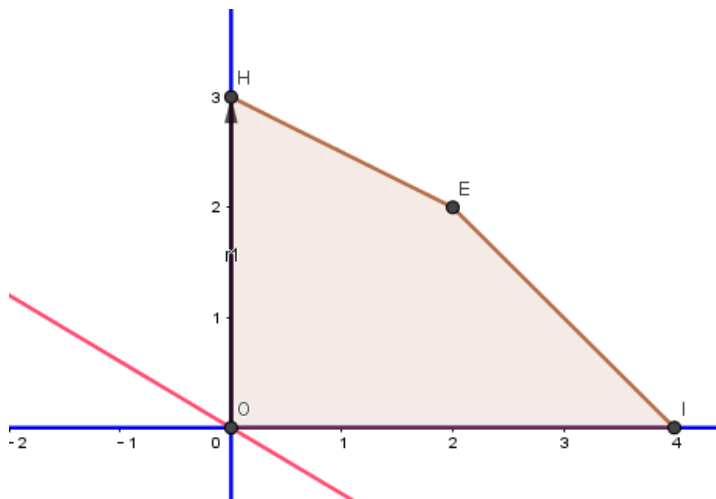
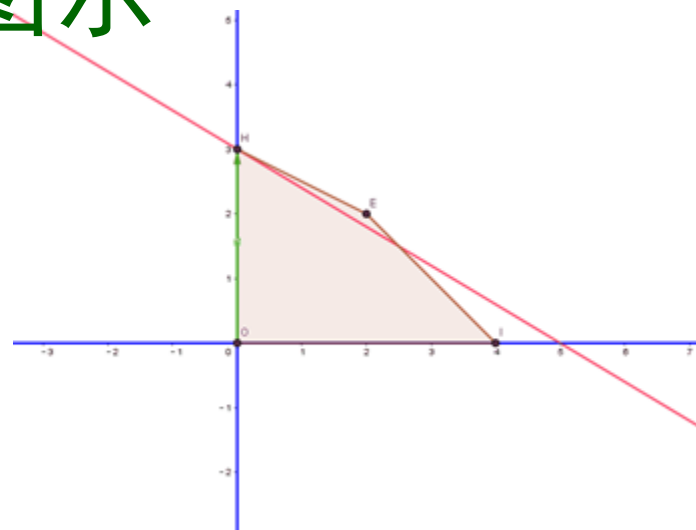
基本可行解V 约束条件中某 n 个约束以等号满足的可行解(可行域极点)

8.1.2 线性规划基本定理(回顾)

- **线性规划基本定理：** 如果线性规划问题有最优解，则必有一基本可行最优解。
- 上述定理的重要意义在于：它把一个最优化问题转化为一个组合问题，即在(8.2) -(8.5)式的 $m+n$ 个约束条件中，确定最优解应满足其中哪 n 个约束条件的问题。
- 由此可知，只要对各种不同的组合进行测试，并比较每种情况下的目标函数值，直到找到最优解。

线性规划单纯形算法图示

$$\begin{aligned} \max \quad & 3x + 5y \\ \text{s.t.} \quad & \begin{cases} x + y \leq 4 \\ x + 2y \leq 6 \\ x \geq 0, y \geq 0 \end{cases} \end{aligned}$$



求解的基本思想：沿着使得目标函数值增加的方向搜索最优解

8.1.3 约束标准型线性规划问题的单纯形算法

- 当线性规划问题中只有等式约束和变量非负约束时，称该线性规划问题具有**标准形式**。
- 先考察一类更特殊的**标准形式线性规划问题**，其中每一个等式约束至少有一个变量的系数为正，且这个变量只在该约束中出现。

$$\begin{array}{ll}\max & z = -x_2 + 3x_3 - 2x_5 \\ \text{s.t.} & x_1 + 3x_2 - x_3 + 2x_5 = 7 \\ & x_4 - 2x_2 + 4x_3 = 12 \\ & x_6 - 4x_2 + 3x_3 + 8x_5 = 10 \\ & x_i \geq 0 \quad i = 1, 2, 3, 4, 5, 6\end{array}$$

约束标准型线性规划问题的单纯形算法

- 在每一约束方程中选择一个这样的变量，并以它作为变量求解该约束方程。这样选出来的变量称为左端变量或**基本变量**，其总数为**m**个。剩下的n-m个变量称为右端变量或**非基本变量**。
- 这一类特殊的标准形式线性规划问题称为**约束标准型线性规划问题**。1.目标函数为求最大值；2.约束条件为等式；3.约束条件右端常数大于等于0；4.变量非负
- 任意一个线性规划问题可以转换为这类问题。

$$\begin{array}{ll}\max & z = -x_2 + 3x_3 - 2x_5 \\ \text{s.t.} & x_1 + 3x_2 - x_3 + 2x_5 = 7 \\ & x_4 - 2x_2 + 4x_3 = 12 \\ & x_6 - 4x_2 + 3x_3 + 8x_5 = 10 \\ & x_i \geq 0 \quad i = 1, 2, 3, 4, 5, 6\end{array}$$

$$\max \quad z = -x_2 + 3x_3 - 2x_5$$

$$\text{s.t.} \quad x_1 + 3x_2 - x_3 + 2x_5 = 7$$

$$x_4 - 2x_2 + 4x_3 = 12$$

$$x_6 - 4x_2 + 3x_3 + 8x_5 = 10$$

$$x_i \geq 0 \quad i = 1, 2, 3, 4, 5, 6$$

单纯形表

非基变量

基本变量

		x_2	x_3	x_5
z		-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

- 任何约束标准型线性规划问题，只要将**所有非基本变量都置为0**，从约束方程式中解出满足约束的基本变量的值，可求得一个基本可行解。
- **单纯形算法的基本思想**就是从**一个基本可行解**出发，进行一系列的基本可行解的变换。
- 每次变换将一个非基本变量与一个基本变量互调位置，且保持当前的线性规划问题是一个**与原问题完全等价**的标准线性规划问题。

- 基本可行解

$x = (7, 0, 0, 12, 0, 10)$ 。

		x_2	x_3	x_5
z	0	-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

单纯形算法计算步骤

步骤1：选入基变量。

- 从非基本变量中选择一个，作为入基变量。

步骤2：选离基变量。

- 选取一个基本变量，为离基变量。

步骤3：转轴变换。

$$\max \quad z = -x_2 + 3x_3 - 2x_5$$

$$\text{s.t.} \quad x_1 + 3x_2 - x_3 + 2x_5 = 7$$

步骤4：转步骤1。

$$x_4 - 2x_2 + 4x_3 = 12$$

$$x_6 - 4x_2 + 3x_3 + 8x_5 = 10$$

$$x_i \geq 0 \quad i = 1, 2, 3, 4, 5, 6$$

- **单纯形算法的第1步：**选出使目标函数增加的非基本变量作为**入基变量**。
- 查看单纯形表的第1行（也称之为z行）中标有非基本变量的各列中的值。选出使目标函数增加的非基本变量作为入基变量。z行中的正系数非基本变量都满足要求。
- z行中只有1列为正，即非基本变量相应的列，其值为3。

$$\max \quad z = -x_2 + 3x_3 - 2x_5$$

- 选取非基本变量 x_3 作为入基变量。

		x_2	x_3	x_5
z	0	-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

- **单纯形算法的第2步：选取离基变量。**
- 在单纯形表中考察由第1步选出的入基变量所相应的列。
- 判断入基变量可以增到多大
 - 如果入基变量所在的列与基本变量所在行交叉处的表元素为负数，那么该元素将不受任何限制，相应的基本变量只会越变越大。
 - 如果入基变量所在列的所有元素都是负值，则**目标函数无界**，已经得到了问题的无界解。
 - 如果选出的列中有一个或多个元素为正数，要**弄清是哪个数限制了入基变量值的增加**。

基变量值的增加。

				x_2	x_3	x_5	
max	$z = -x_2 + 3x_3 - 2x_5$	z		0	-1	3	-2
	$x_1 + 3x_2 - x_3 + 2x_5 = 7$	x_1		7	3	-1	2
	$x_4 - 2x_2 + 4x_3 = 12$	x_4		12	-2	4	0
	$x_6 - 4x_2 + 3x_3 + 8x_5 = 10$	x_6		10	-4	3	8
	$x_i \geq 0 \quad i = 1, 2, 3, 4, 5, 6$						

- 受限的增加量可以用入基变量所在列的元素（称为主元素）来除主元素所在行的“常数列”（最左边的列）中元素而得到。所得到数值越小说明受到限制越多。
- 应该选取受到限制最多的基本变量作为离基变量，才能保证将入基变量与离基变量互调位置后，仍满足约束条件。
- 上例中，惟一的一个值为正的z行元素是3，它所在列中有2个正元素，即4和3。
- $\min\{12/4, 10/3\}=3$ ，应该选取 x_4 为离基变量； x_2 x_3 x_5
- 入基变量 x_3 取值为3。

	z	x_2	x_3	x_5
	0	-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

- **单纯形算法的第3步：转轴变换。**
- 转轴变换的目的是将入基变量与离基变量互调位置。
- 给入基变量一个增值，使之成为基本变量；
- 修改离基变量，让入基变量所在列中，离基变量所在行的元素值减为零，而使之成为非基本变量。

- 解离基变量所相应的方程，将入基变量 x_3 用离基变量 x_4 表示为 $x_3 - \frac{1}{2}x_2 + \frac{1}{4}x_4 = 3$

- 再将其代入其他基本变量和所在的行中消去 x_3 ，

$$x_1 + \frac{5}{2}x_2 + \frac{1}{4}x_4 + 2x_5 = 10$$

$$x_6 - \frac{5}{2}x_2 - \frac{3}{4}x_4 + 8x_5 = 1$$

基本可行解

$$x = (10, 0, 3, 0, 0, 1)$$

- 代入目标函数得到

$$z = 9 + \frac{1}{2}x_2 - \frac{3}{4}x_4 - 2x_5$$

- 形成新单纯形表

		x_2	x_4	x_5
z	9	1/2	-3/4	-2
x_1	10	5/2	1/4	2
x_3	3	-1/2	1/4	0
x_6	1	-5/2	-3/4	8

- **单纯形算法的第4步：**转回并重复第1步，进一步改进目标函数值。
- 不断重复上述过程，直到z行的所有非基本变量系数都变成负值为止。这表明目标函数不可能再增加了。
- 单纯形表中，唯一的值为正的z行元素是非基本变量 x_2 相应的列，其值为1/2。因此，选取非基本变量 x_2 作为入基变量。
- 它所在列中有唯一的正元素5/2，即基本变量 x_1 相应行的元素
- 因此，选取 x_1 为离基变量。

		x_2	x_4	x_5
z	9	1/2	-3/4	-2
x_1	10	5/2	1/4	2
x_3	3	-1/2	1/4	0
x_6	1	-5/2	-3/4	8

- 再经步骤3的转轴变换得到新单纯形表。新单纯形表z行的所有非基本变量系数都变成负值，求解过程结束。整个问题的解可以从最后一张单纯形表的常数列中读出。

- 目标函数 $z=11+(-1/5)x_1+(-4/5)x_4+(-12/5)x_5$,
最优值为11;

- 最优解为: (0,4,5,0,0,11)

		x_1	x_4	x_5
z	11	-1/5	-4/5	-12/5
x_2	4	2/5	1/10	4/5
x_3	5	1/5	3/10	2/5
x_6	11	1	-1/2	10

基本可行解

$$x = (7, 0, 0, 12, 0, 10)$$

		x_2	x_4	x_5
z	9	$1/2$	$-3/4$	-2
x_1	10	$5/2$	$1/4$	2
x_3	3	$-1/2$	$1/4$	0
x_6	1	$-5/2$	$-3/4$	8

基本可行解

$$x = (10, 0, 3, 0, 0, 1)$$

		x_2	x_3	x_5
z	0	-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

最优解为: $x^* = (0, 4, 5, 0, 0, 11)$

		x_1	x_4	x_5
z	11	$-1/5$	$-4/5$	$-12/5$
x_2	4	$2/5$	$1/10$	$4/5$
x_3	5	$1/5$	$3/10$	$2/5$
x_6	11	1	$-1/2$	10

单纯形算法计算步骤

步骤1：选入基变量。

- 如果所有 $c_j \leq 0$ ，则当前基本可行解为最优解，**计算结束**。
- 否则取 $c_e > 0$ 相应的非基本变量 x_e 为入基变量。

步骤2：选离基变量。

- 对于步骤1选出的入基变量 x_e ，如果所有 $a_{ie} \leq 0$ ，则最优解无界，**计算结束**。
- 否则计算 $\theta = \min_{a_{ie} > 0} \left\{ \frac{b_i}{a_{ie}} \right\} = \frac{b_k}{a_{ke}}$
- 选取基本变量 x_k 为离基变量。
- 新的基本变量下标集为 $\bar{B} = B + \{e\} - \{k\}$
- 新的非基本变量下标集为 $\bar{N} = N + \{k\} - \{e\}$

■步骤3：转轴变换。

■新单纯形表中各元素变换如下。

线性代数中的高斯消元法，调整基变量，同时保证可行性和目标函数优化。

$$\left\{ \begin{array}{l} \bar{b}_i = b_i - a_{ie} \frac{b_k}{a_{ke}} \quad i \in \bar{B} \\ \bar{b}_e = \frac{b_k}{a_{ke}} \\ \bar{a}_{ij} = a_{ij} - a_{ie} \frac{a_{kj}}{a_{ke}} \quad i \in \bar{B}, \quad j \in \bar{N} \\ \bar{a}_{ik} = -\frac{a_{ie}}{a_{ke}} \\ \bar{a}_{ej} = \frac{a_{kj}}{a_{ke}} \quad j \in \bar{N} \\ \bar{a}_{ek} = \frac{1}{a_{ke}} \\ \bar{c}_i = c_i - c_e \frac{a_{ki}}{a_{ke}} \quad i \in \bar{N} \\ \bar{c}_k = -\frac{c_e}{a_{ke}} \end{array} \right.$$

■步骤4：转步骤1。

8.1.4 将一般问题转化为约束标准型

■ 转化方法：

- 首先，需要把(8.2)或(8.4)形式的不等式约束转换为等式约束。
- 具体做法是，引入**松弛变量**，利用松弛变量的非负性，将不等式转化为等式。松弛变量记为 y_i ，共有 m_1+m_3 个。
- 在求解过程中，应当将松弛变量与原来变量同样对待。求解结束后，抛弃松弛变量。注意**松弛变量前的符号由相应的原不等式的方向所确定**。

$$\begin{aligned}x_1 + 2x_3 &\leq 18 \\2x_2 - 7x_4 &\leq 0 \\x_1 + x_2 + x_3 + x_4 &= 9 \\x_2 - x_3 + 2x_4 &\geq 1\end{aligned}$$

$$\begin{aligned}x_1 + 2x_3 + y_1 &= 18 \\2x_2 - 7x_4 + y_2 &= 0 \\x_1 + x_2 + x_3 + x_4 &= 9 \\x_2 - x_3 + 2x_4 - y_3 &= 1\end{aligned}$$

- 为了进一步构造标准型约束，还需要引入m个**人工变量**，记为 z_i 。人工变量也要满足非负约束。
- 至此，原问题已变换为等价的约束标准型线性规划问题
- 极小化线性规划问题，只要将目标函数乘以-1即可化为等价的极大化线性规划问题。

$$z_1 + x_1 + 2x_3 + y_1 = 18$$

$$z_2 + 2x_2 - 7x_4 + y_2 = 0$$

$$z_3 + x_1 + x_2 + x_3 + x_4 = 9$$

$$z_4 + x_2 - x_3 + 2x_4 - y_3 = 1$$

8.1.5一般线性规划问题的2阶段单纯形算法

- 引入人工变量后的线性规划问题与原问题并不等价，除非所有 z_i 都是0。为了解决这个问题，在求解时必须分2个阶段进行。
- **第一阶段**用一个辅助目标函数 $z' = -\sum_{i=1}^m z_i$ 替代原来的目标函数。
- 这个线性规划问题称为原线性规划问题所相应的**辅助线性规划问题**，对辅助线性规划问题用单纯形算法求解。
- 如果原线性规划问题有可行解，则辅助线性规划问题就有最优解，且其最优值为0，即所有 z_i 都为0。

8.1.5一般线性规划问题的2阶段单纯形算法

- 在辅助线性规划问题最后的单纯形表中，所有 z_i 均为非基本变量。划掉所有 z_i 相应的列，剩下的就是只含 x_i 和 y_i 的约束标准型线性规划问题，且与原问题等价。
- 单纯形算法第一阶段的任务就是构造一个初始基本可行解。
- 单纯形算法第二阶段的目标是求解由第一阶段导出的问题。此时要用原来的目标函数进行求解。
- 如果在辅助线性规划问题最后的单纯形表中， z_i 不全为0，则原线性规划问题没有可行解，从而原线性规划问题无解。

8.1.5一般线性规划问题的2阶段单纯形算法

示例: 问题转化

$$\begin{aligned} \max Z &= -3x_1 + x_3 \\ \text{s.t. } \begin{cases} x_1 + x_2 + x_3 \leq 4 \\ -2x_1 + x_2 - x_3 \geq 1 \\ 3x_2 + x_3 = 9 \\ x_1, x_2, x_3 \geq 0 \end{cases} \end{aligned}$$

0.1) 将其转化为标准形式,
加入两个松弛变量 x_4 、 x_5

$$\begin{aligned} \max Z &= -3x_1 + x_3 + 0x_4 + 0x_5 \\ \text{s.t. } \begin{cases} x_1 + x_2 + x_3 + x_4 = 4 \\ -2x_1 + x_2 - x_3 - x_5 = 1 \\ 3x_2 + x_3 = 9 \\ x_1, x_2, x_3, x_4, x_5 \geq 0 \end{cases} \end{aligned}$$

0.2) 加入两个人工变量 x_6 、 x_7

$$\begin{aligned} \max Z &= -3x_1 + x_3 + 0x_4 + 0x_5 \\ \text{s.t. } \begin{cases} x_1 + x_2 + x_3 + x_4 = 4 \\ -2x_1 + x_2 - x_3 - x_5 + x_6 = 1 \\ 3x_2 + x_3 + x_7 = 9 \\ x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0 \end{cases} \end{aligned}$$

8.1.5一般线性规划问题的2阶段单纯形算法

示例: 第一阶段

$$= 2x_1 - 4x_2 + x_5 + 10$$

构造辅助函数

$$\min Z = x_6 + x_7$$

列表求解

$$s.t. \begin{cases} x_1 + x_2 + x_3 + x_4 = 4 \\ -2x_1 + x_2 - x_3 - x_5 + x_6 = 1 \\ 3x_2 + x_3 + x_7 = 9 \\ x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0 \end{cases}$$

单纯形表

		x_1	x_2	x_3	x_5
z	10	2	-4	0	1
x_4	4	1	1	1	0
x_6	1	-2	1	-1	-1
x_7	9	0	3	1	0

x_2 入基, x_6 出基

8.1.5一般线性规划问题的2阶段单纯形算法

示例: 第一阶段

		x_1	x_3	x_5	x_6
z	6	-6	-4	-3	4
x_4	3	3	2	1	-1
x_2	1	-2	-1	-1	1
x_7	6	6	4	3	-3

x_1 入基; x_7 出基

人工变量出基, $Z=0$, 表示优化问题有解, 并找到了基本可行解(1,3,0,0,0)

		x_3	x_5	x_6	x_7
z	0	0	0	1	1
x_4	0	0	-1/2	1/2	-1/2
x_2	3	1/3	0	0	1/3
x_1	1	2/3	1/2	-1/2	1/6

8.1.5一般线性规划问题的2阶段单纯形算法

示例: 第二阶段

基本可行解(1,3,0,0,0)

去掉人工变量, 将目标函数的系数换成原问题的目标函数系数, 作为第二阶段计算初始表, 用单纯形法继续计算

$$\max Z = -3x_1 + x_3 + 0x_4 + 0x_5$$

原问题标准型

$$s.t \begin{cases} x_1 + x_2 + x_3 + x_4 = 4 \\ -2x_1 + x_2 - x_3 - x_5 = 1 \\ 3x_2 + x_3 = 9 \\ x_1, x_2, x_3, x_4, x_5 \geq 0 \end{cases}$$

转化后的问题

$$\max Z = 3x_3 + 3/2x_5 - 3$$
$$s.t \begin{cases} x_1 + 2/3x_3 + 1/2x_5 = 1 \\ x_4 - 1/2x_5 = 0 \\ x_2 + 1/3x_3 = 3 \\ x_1, x_2, x_3, x_4, x_5 \geq 0 \end{cases}$$

	x_3	x_5
Z	-3	3/2
x_4	0	-1/2
x_2	3	0
x_1	1	1/2

x_3 入基, x_1 出基

8.1.5一般线性规划问题的2阶段单纯形算法

示例: 第二阶段

去掉人工变量，将目标函数的系数换成原问题的目标函数系数，作为第二阶段计算初始表，用单纯形法继续计算

		x_1	x_5
z	$3/2$	$-9/2$	$-3/4$
x_4	0	0	$-1/2$
x_2	$5/2$	$-1/2$	$1/4$
x_3	$3/2$	$3/2$	$3/4$

此时第一行第二列和第三列已经没有正数

最优解 $(0, 5/2, 3/2, 0, 0)$

最优值: $3/2$

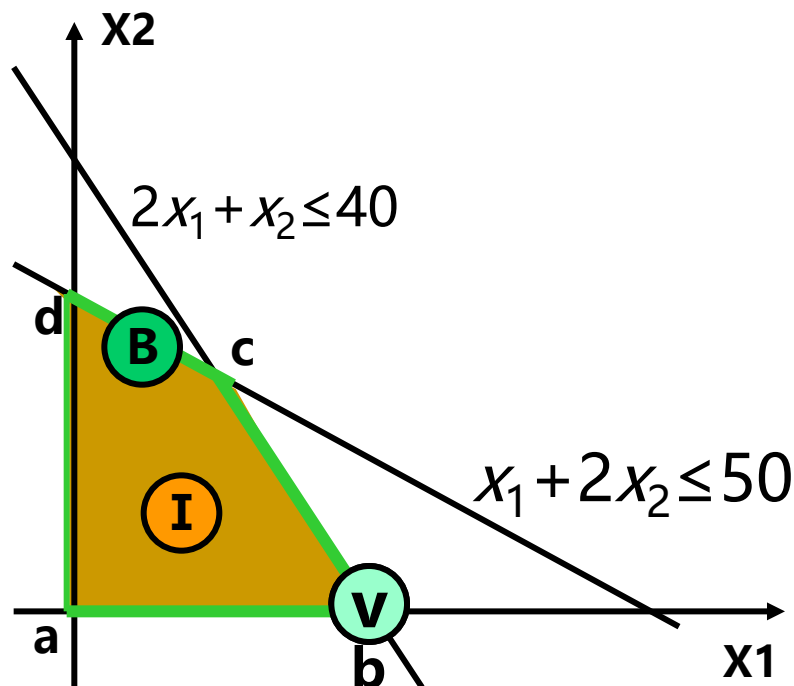
两阶段单纯形算法

- 约束标准型问题的求解基本步骤？
- 入基变量的选择依据？
- 出基变量的选择依据？
- 转轴变换起到的作用是什么？
- 一般线性规划问题的两个阶段目的分别是什么？
- 问题转化时引入两类变量分别是？
- **单纯性算法的数学基础是什么？**

单纯性算法的数学基础

(1) 几何学

- 线性规划的可行域是由线性不等式定义的凸多面体
- 顶点对应基本可行解
- 单纯形法从一个顶点出发，沿着凸多面体的边移动到相邻顶点，始终在凸集边界上搜索



$$\begin{aligned} \max \quad & z = -x_2 + 3x_3 - 2x_5 \\ & x_1 + 3x_2 - x_3 + 2x_5 = 7 \\ & x_4 - 2x_2 + 4x_3 = 12 \\ & x_6 - 4x_2 + 3x_3 + 8x_5 = 10 \end{aligned}$$

单纯性算法的数学基础

■ (2) 优化方向选择 (检验数)

从一个可行解换到另一个更好的解

选入基变量： 检验数大于0的变量中选最大/随机选/序号最小

选出基变量： 用最小比值测试决定谁离开。

$$\begin{aligned}\max \quad & z = -x_2 + 3x_3 - 2x_5 \\ & x_1 + 3x_2 - x_3 + 2x_5 = 7 \\ & x_4 - 2x_2 + 4x_3 = 12 \\ & x_6 - 4x_2 + 3x_3 + 8x_5 = 10\end{aligned}$$

		x_2	x_3	x_5
z	0	-1	3	-2
x_1	7	3	-1	2
x_4	12	-2	4	0
x_6	10	-4	3	8

单纯性算法的数学基础

■ (3) 线性代数与基变换

线性代数中的高斯消元法，调整基变量，同时保证可行性和目标函数优化。

$$\max \quad z = -x_2 + 3x_3 - 2x_5$$

$$x_1 + 3x_2 - x_3 + 2x_5 = 7$$

$$x_4 - 2x_2 + 4x_3 = 12$$

$$x_6 - 4x_2 + 3x_3 + 8x_5 = 10$$

$$z = 9 + \frac{1}{2}x_2 - \frac{3}{4}x_4 - 2x_5$$

$$x_1 + \frac{5}{2}x_2 + \frac{1}{4}x_4 + 2x_5 = 10$$

$$x_6 - \frac{5}{2}x_2 - \frac{3}{4}x_4 + 8x_5 = 1$$

$$x_3 - \frac{1}{2}x_2 + \frac{1}{4}x_4 = 3$$

学习要点

- 理解线性规划算法模型
- 掌握解线性规划问题的单纯形算法
- **最大网络流问题**
- 增广路算法

8.2 最大网络流问题

许多实际系统都涉及到流量问题

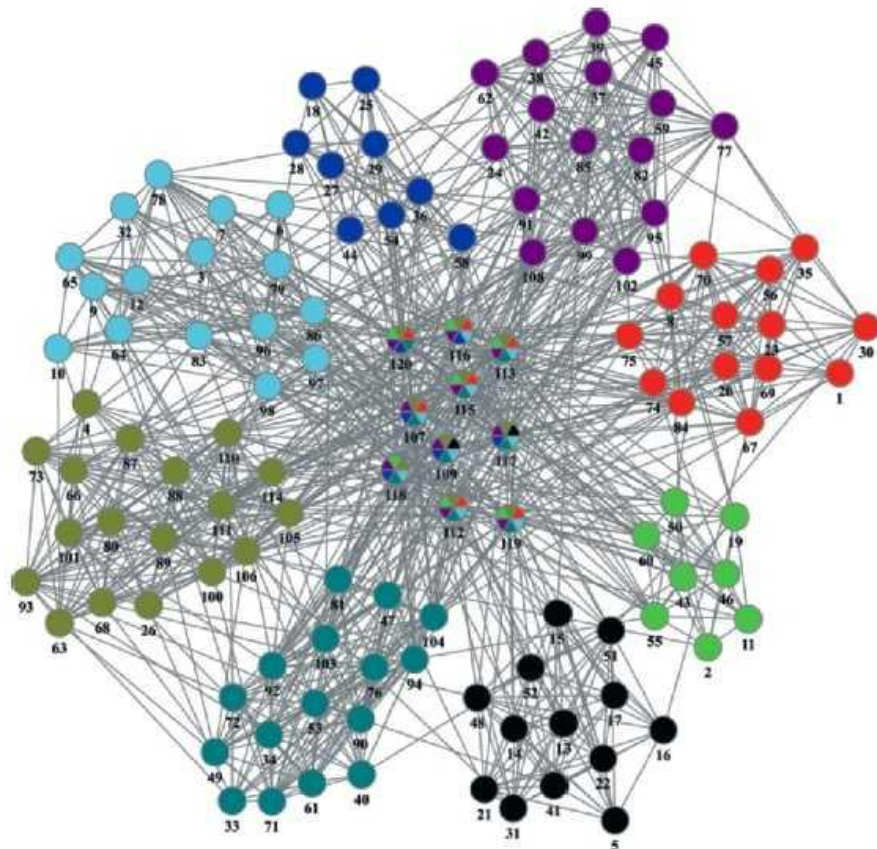
网络系统中有信息流

交通系统中有车辆流

金融系统中有现金流

.....

最大流量?



例如一个交通网络，结点表示车站，边表示道路，边权表示两个车站间道路的通行能力。给定一个网络，要求指定两点间的最大车流量，即最大流问题。

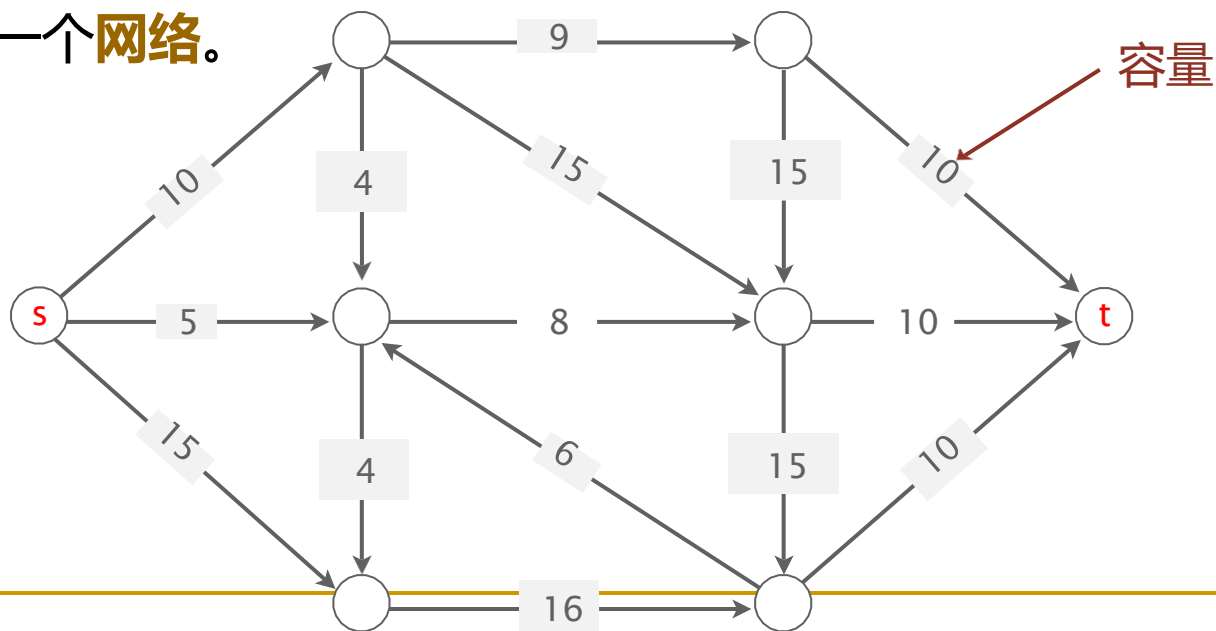
8.2 最大网络流问题

1 基本概念和术语

(1) 网络

G 是一个简单有向图, $G=(V,E)$, $V=\{1, 2, \dots, n\}$ 。在 V 中指定一个顶点 s 称为**源**, 和另一个顶点 t 称为**汇**。有向图 G 的每一条边 $(v,w) \in E$, 对应有一个值 $\text{cap}(v,w) \geq 0$, 称为边的**容量**。

这样的有向图 G 称作一个**网络**。

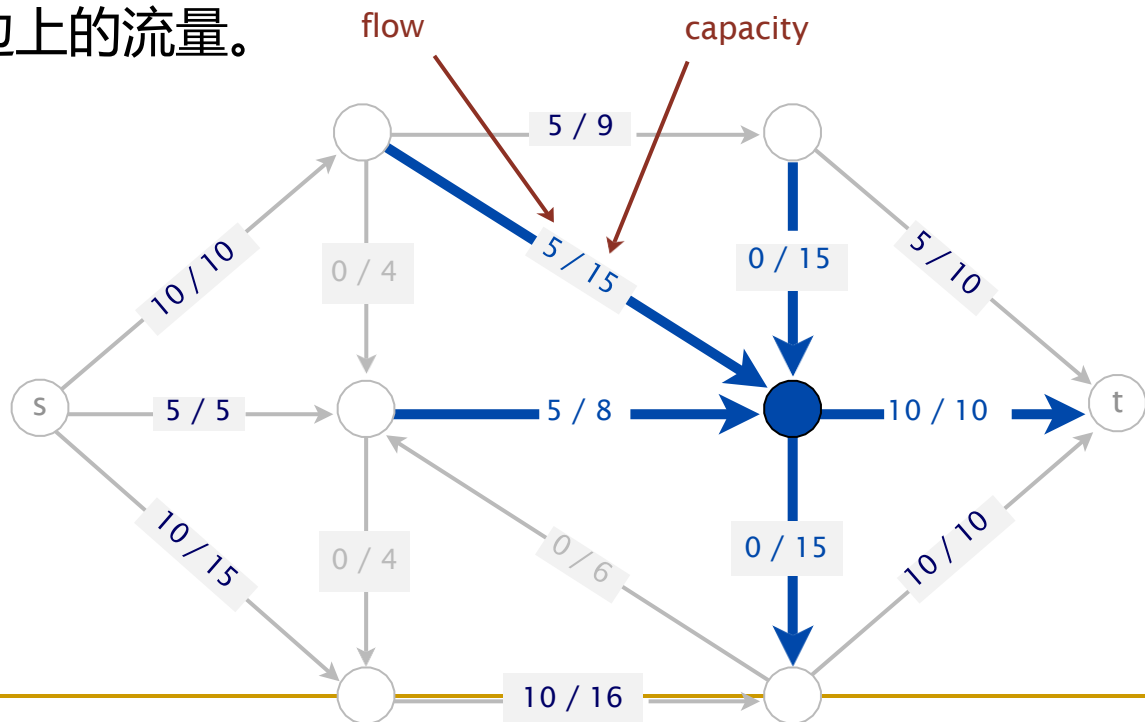


8.2 最大网络流问题

1 基本概念和术语

(2) 网络流

网络上的**流**是定义在网络的边集合 E 上的一个**非负**函数 $\text{flow}=\{\text{flow}(v,w)\}$ ，并称 $\text{flow}(v,w)$ 为边 (v,w) 上的**流量**。假定有 m 条边，一个**网络流** $(x_1, x_2, \dots, x_m)$ ，其中 x_i 是第 i 条边上的流量。

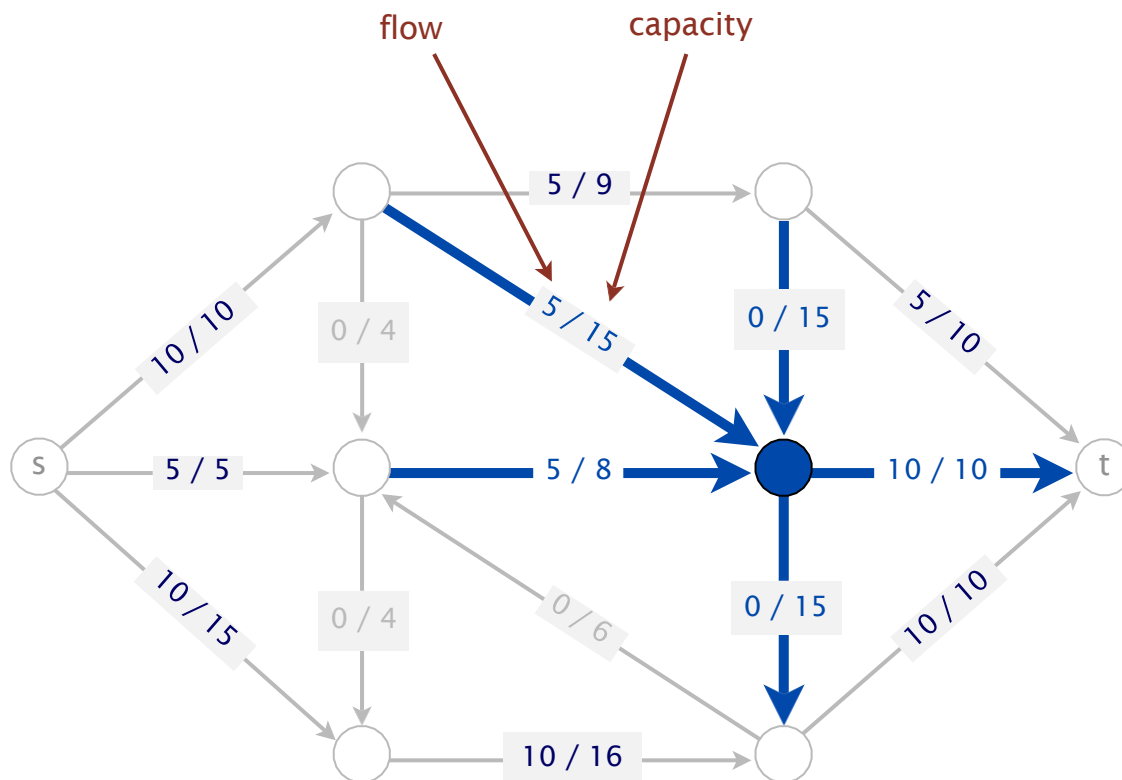


(3) 可行流

满足下述条件的流flow称为可行流:

(3.1) 容量约束: 对每一条边 $(v,w) \in E$, $0 \leq \text{flow}(v,w) \leq \text{cap}(v,w)$ 。

(3.2) 平衡约束



(3) 可行流

满足下述条件的流flow称为**可行流**:

(3.1)容量约束: 对每一条边 $(v,w) \in E$, $0 \leq \text{flow}(v,w) \leq \text{cap}(v,w)$ 。

(3.2)平衡约束:

对于中间顶点: 流出量=流入量, $\sum_{(v,w) \in E} \text{flow}(v,w) - \sum_{(w,v) \in E} \text{flow}(w,v) = 0$

对于源s: s的流出量 - s的流入量=源的净输出量f

$$\sum_{(s,v) \in E} \text{flow}(s,v) - \sum_{(v,s) \in E} \text{flow}(v,s) = f$$

对于汇t: t的流入量 - t的流出量的=汇的净输入量f

$$\sum_{(v,t) \in E} \text{flow}(v,t) - \sum_{(t,v) \in E} \text{flow}(t,v) = f$$

式中f 称为这个可行流的流量, 即源的净输出量(或汇的净输入量)。

可行流总是存在的。例如, 所有边的流量 $\text{flow}(v,w)=0$, 就得到一个流量f=0的可行流(称为0流)

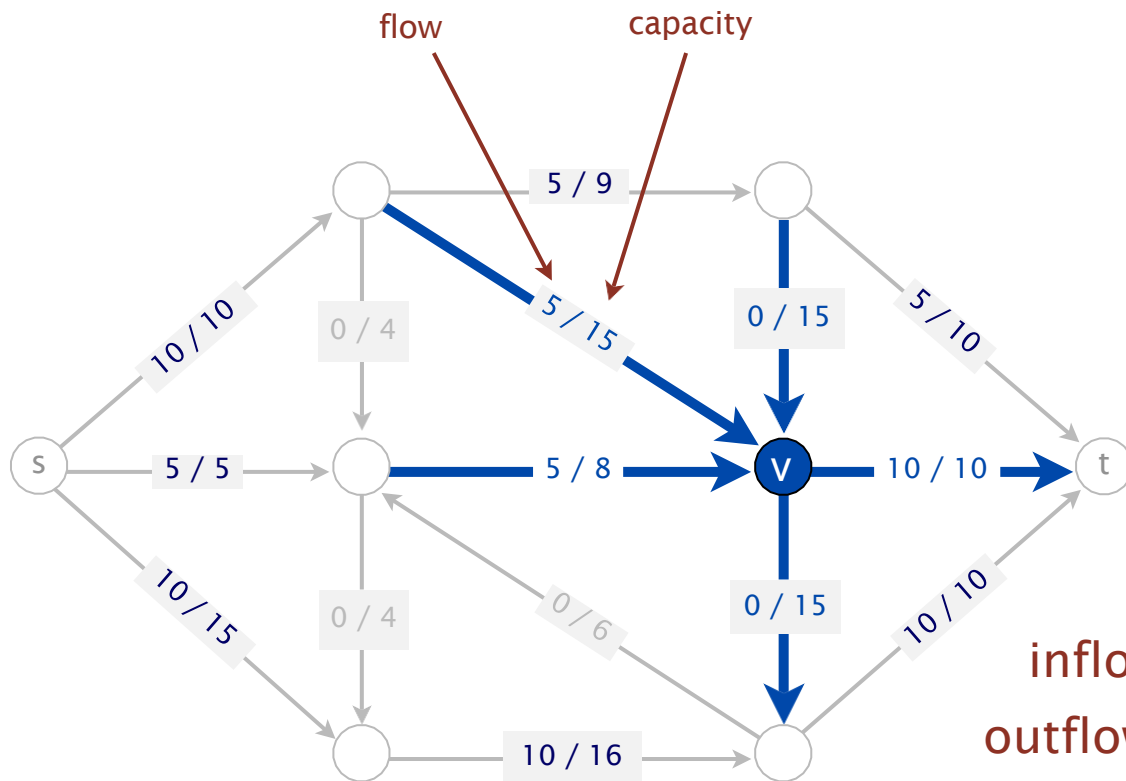
(3) 可行流

满足下述条件的流flow称为**可行流**:

(3.1)容量约束: 对每一条边 $(v,w) \in E$, $0 \leq \text{flow}(v,w) \leq \text{cap}(v,w)$ 。

(3.2)平衡约束:

对于中间顶点:
$$\sum_{(v,w) \in E} \text{flow}(v,w) - \sum_{(w,v) \in E} \text{flow}(w,v) = 0$$



inflow at $v = 5 + 5 + 0 = 10$
outflow at $v = 10 + 0 = 10$

(3) 可行流

满足下述条件的流flow称为**可行流**:

(3.1)容量约束: 对每一条边 $(v,w) \in E$, $0 \leq \text{flow}(v,w) \leq \text{cap}(v,w)$ 。

(3.2)平衡约束:

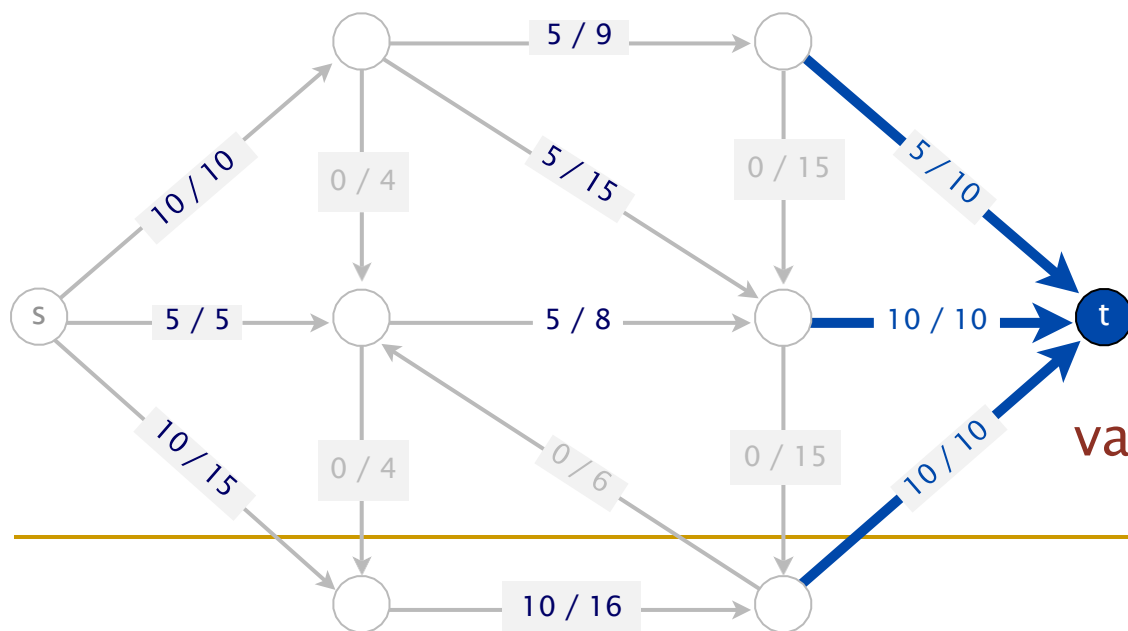
对于中间顶点:

$$\sum_{(v,w) \in E} \text{flow}(v,w) - \sum_{(w,v) \in E} \text{flow}(w,v) = 0$$

对于源s:

$$\sum_{(s,v) \in E} \text{flow}(s,v) - \sum_{(v,s) \in E} \text{flow}(v,s) = f$$

对于汇t:

$$\sum_{(v,t) \in E} \text{flow}(v,t) - \sum_{(t,v) \in E} \text{flow}(t,v) = f$$


$$\text{value} = 5 + 10 + 10 = 25$$

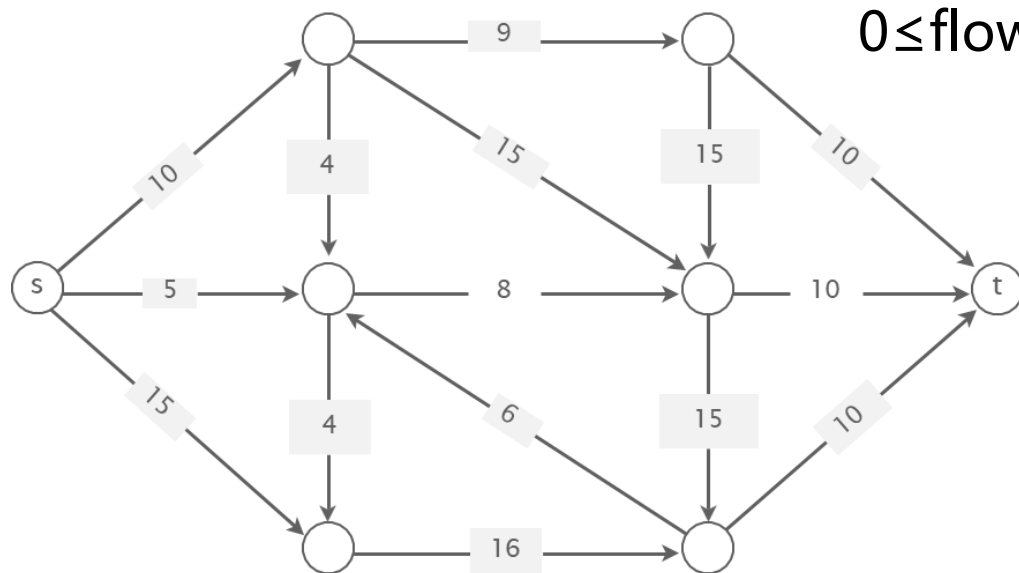
(4) 网络最大流问题

最大流问题即求网络G的一个可行流flow, 使其流量f达到最大。

表示为如下线性规划问题:

$$\max f$$

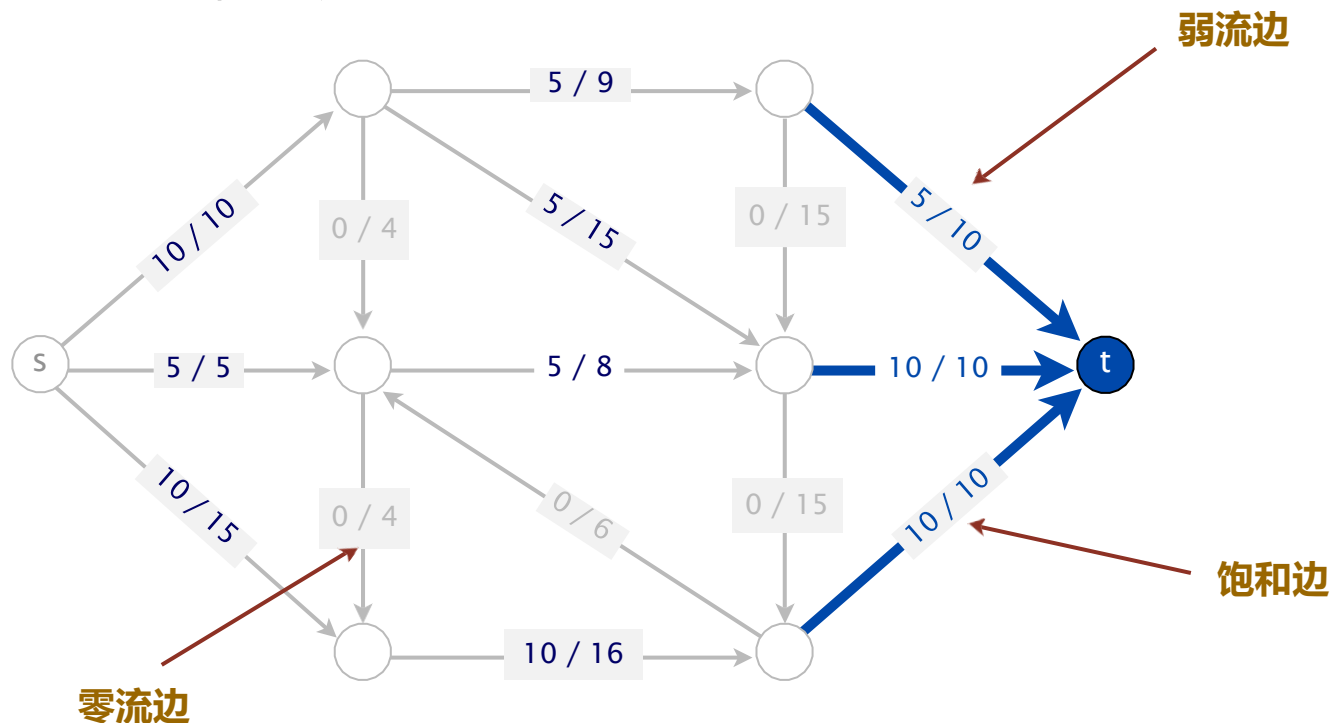
$$\text{s.t.} \quad \sum flow(v, w) - \sum flow(w, v) = \begin{cases} f & v = s \\ 0 & v \neq s, t \\ -f & v = t \end{cases}$$



$$0 \leq flow(v, w) \leq cap(v, w), (v, w) \in E$$

(5) 边流

对于网络 G 的一个给定的可行流，将网络中满足 $\text{flow}(v,w)=\text{cap}(v,w)$ 的边称为**饱和边**； $\text{flow}(v,w)<\text{cap}(v,w)$ 的边称为**非饱和边**； $\text{flow}(v,w)=0$ 的边称为**零流边**； $\text{flow}(v,w)>0$ 的边称为**非零流边**。当边 (v,w) 既不是一条零流边也不是一条饱和边时，称为**弱流边**。



(6) 残流网络

残流网络是设计与网络流有关算法的重要工具。

对于给定的一个流网络 G 及其上的一个流 flow ，网络 G 关于流 flow 的残流网络 G^* 与 G 有相同的顶点集 V ，而网络 G 中的每一条边对应于 G^* 中的1条边或2条边。

设 (v,w) 是 G 的一条边，

- 当 $\text{flow}(v,w) > 0$ 时， (w,v) 是 G^* 中的一条边，该边的容量为
 $\text{cap}^*(w,v) = \text{flow}(v,w)$;
- 当 $\text{flow}(v,w) < \text{cap}(v,w)$ 时， (v,w) 是 G^* 中的一条边，该边的容量为
 $\text{cap}^*(v,w) = \text{cap}(v,w) - \text{flow}(v,w)$

(6) 残流网络

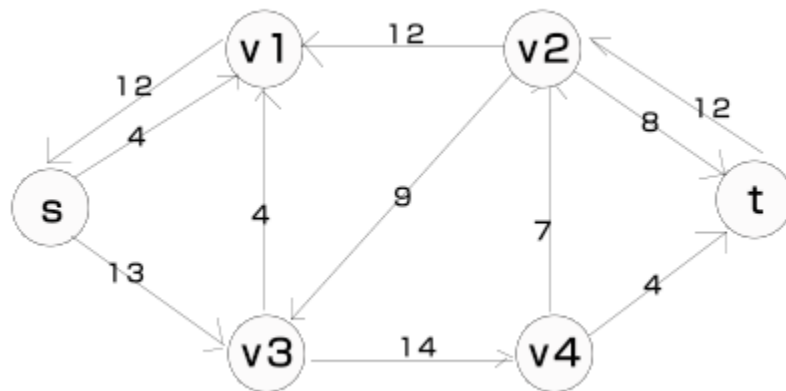
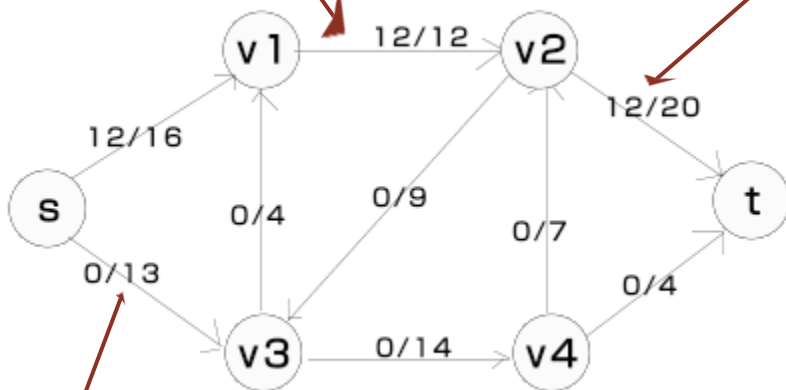
残流网络是设计与网络流有关算法的重要工具。残流网络描述了图中各边的剩余容量以及可以通过“回流”删除的流量大小

饱和边

残流网络 G^* 中有唯一的1条边 (w,v) 与之对应，且该边的容量为 $\text{cap}(v,w)$

弱流边

残流网络 G^* 中有2条边 (v,w) 和 (w,v) 与之对应，这2条边的容量分别为 $\text{cap}(v,w) - \text{flow}(v,w)$ 和 $\text{flow}(v,w)$



零流边

残流网络 G^* 中有唯一的1条边 (v,w) 与之对应，且该边的容量为 $\text{cap}(v,w)$

学习要点

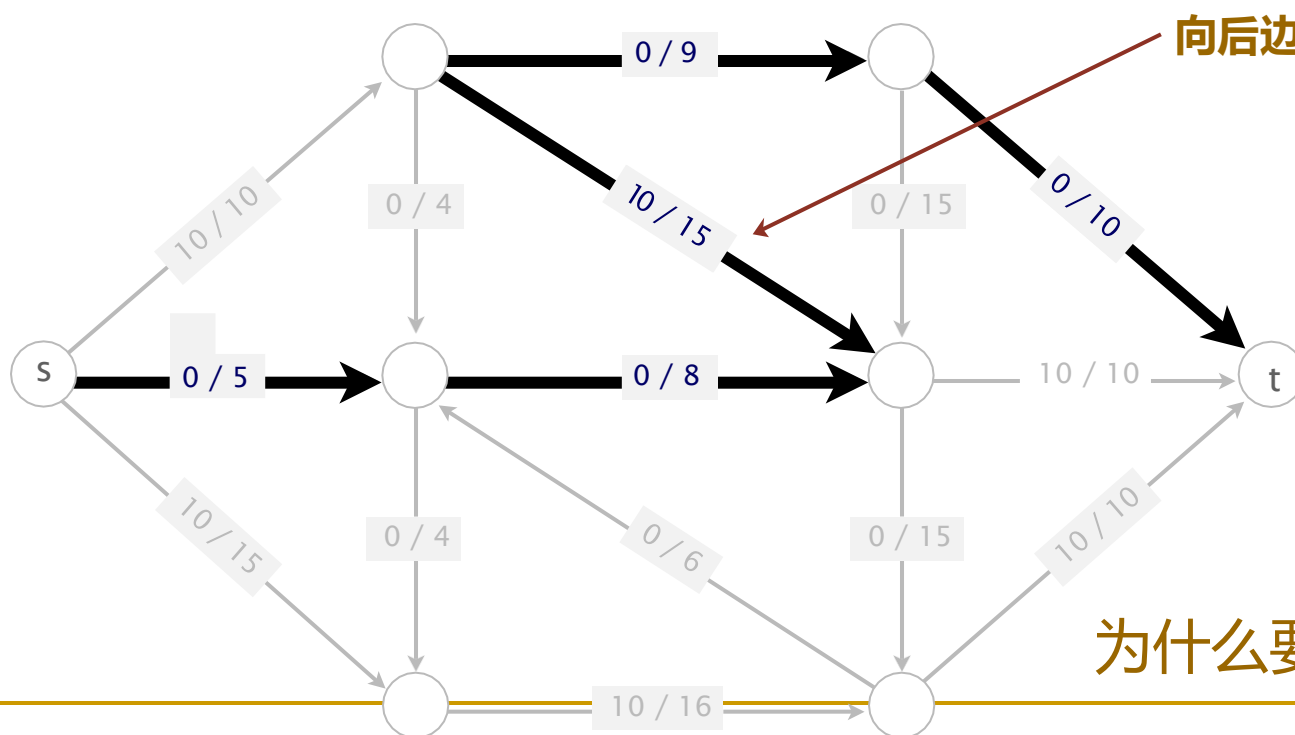
- 理解线性规划算法模型
- 掌握解线性规划问题的单纯形算法
- 最大网络流问题
- **增广路算法**

8.2.2 增广路算法

1. 可增广路

设 P 是网络 G 中联结源 s 和汇 t 的一条路。路的方向从 s 到 t 。路 P 上边分成2类：

- 一类边的方向与路的方向一致，称为**向前边**。向前边的全体记为 P^+
- 另一类边的方向与路的方向相反，称为**向后边**。向后边的全体记为 P^-



为什么要考虑向后边？

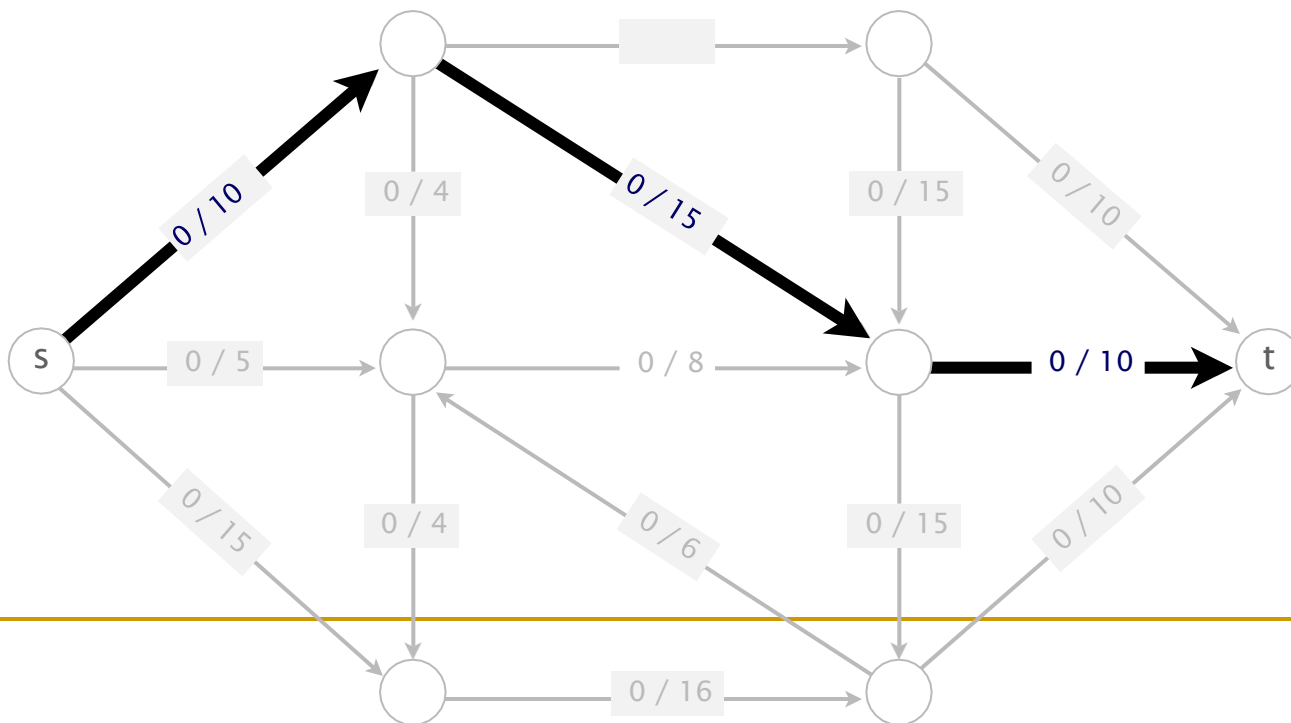
8.2.2 增广路算法

1. 可增广路

设 flow 是一个可行流， P 是从 s 到 t 的一条路，若 P 满足下列条件：

- P 的所有向前边 (v,w) 上， $\text{flow}(v,w) < \text{cap}(v,w)$ ；
- P 的所有向后边 (v,w) 上， $\text{flow}(v,w) > 0$ 。

则称 P 为关于可行流 flow 的一条可增广路。



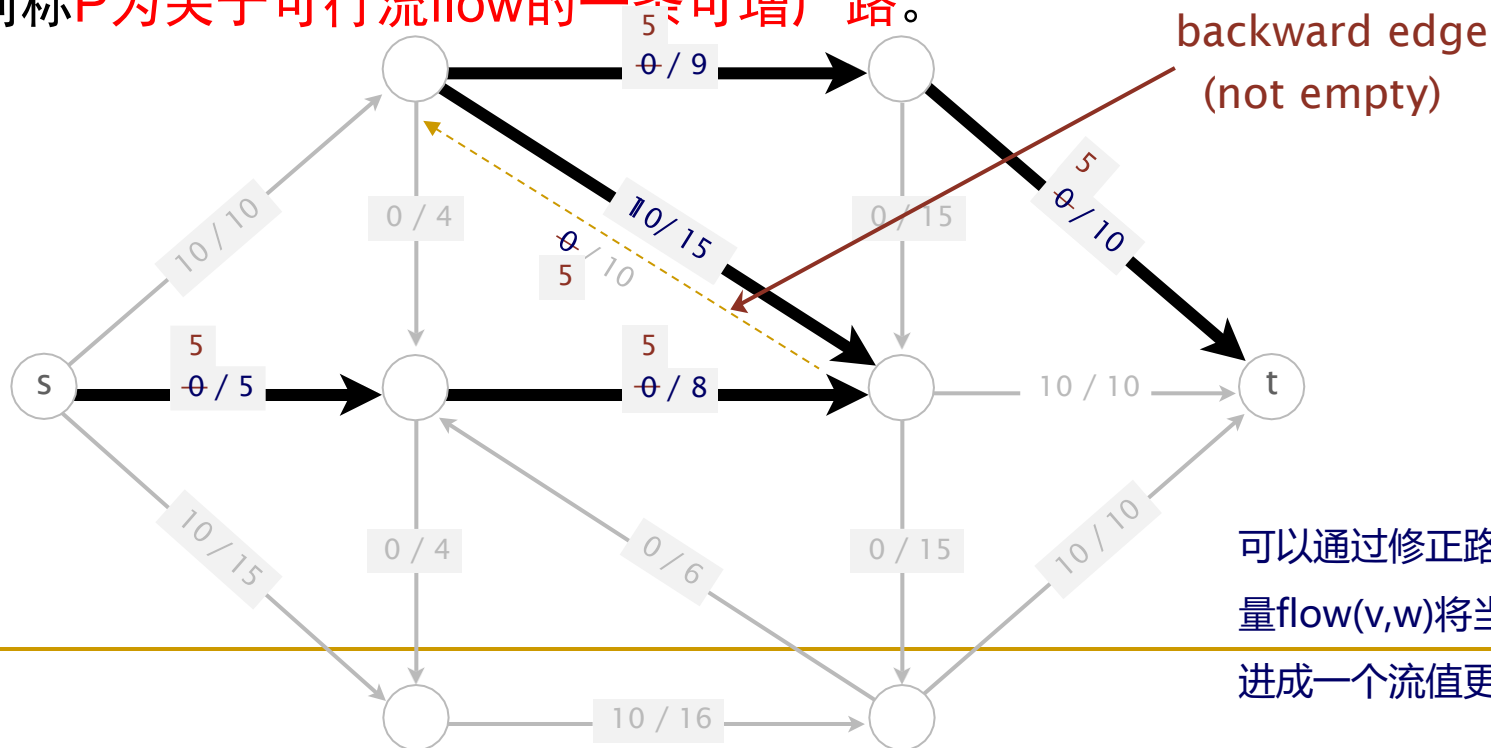
8.2.2 增广路算法

1. 可增广路

设 flow 是一个可行流， P 是从 s 到 t 的一条路，若 P 满足下列条件：

- P 的所有**向前边** (v,w) 上， $\text{flow}(v,w) < \text{cap}(v,w)$ ；
- P 的所有**向后边** (v,w) 上， $\text{flow}(v,w) > 0$ 。

则称 P 为关于可行流 flow 的一条**可增广路**。



2. 增流的具体做法是：

- (1) 不属于可增广路P的边(v,w)上的流量保持不变；
- (2) 可增广路P上的所有边(v,w)上的流量按规则修改当前的流。

$$flow(v, w) = \begin{cases} flow(v, w) + d & (v, w) \in P^+ \\ flow(v, w) - d & (v, w) \in P^- \\ flow(v, w) & (v, w) \notin P \end{cases}$$

其中**d称为可增广量**，可按下述原则确定：d取得尽量大，又要使变化后的流仍为可行流。

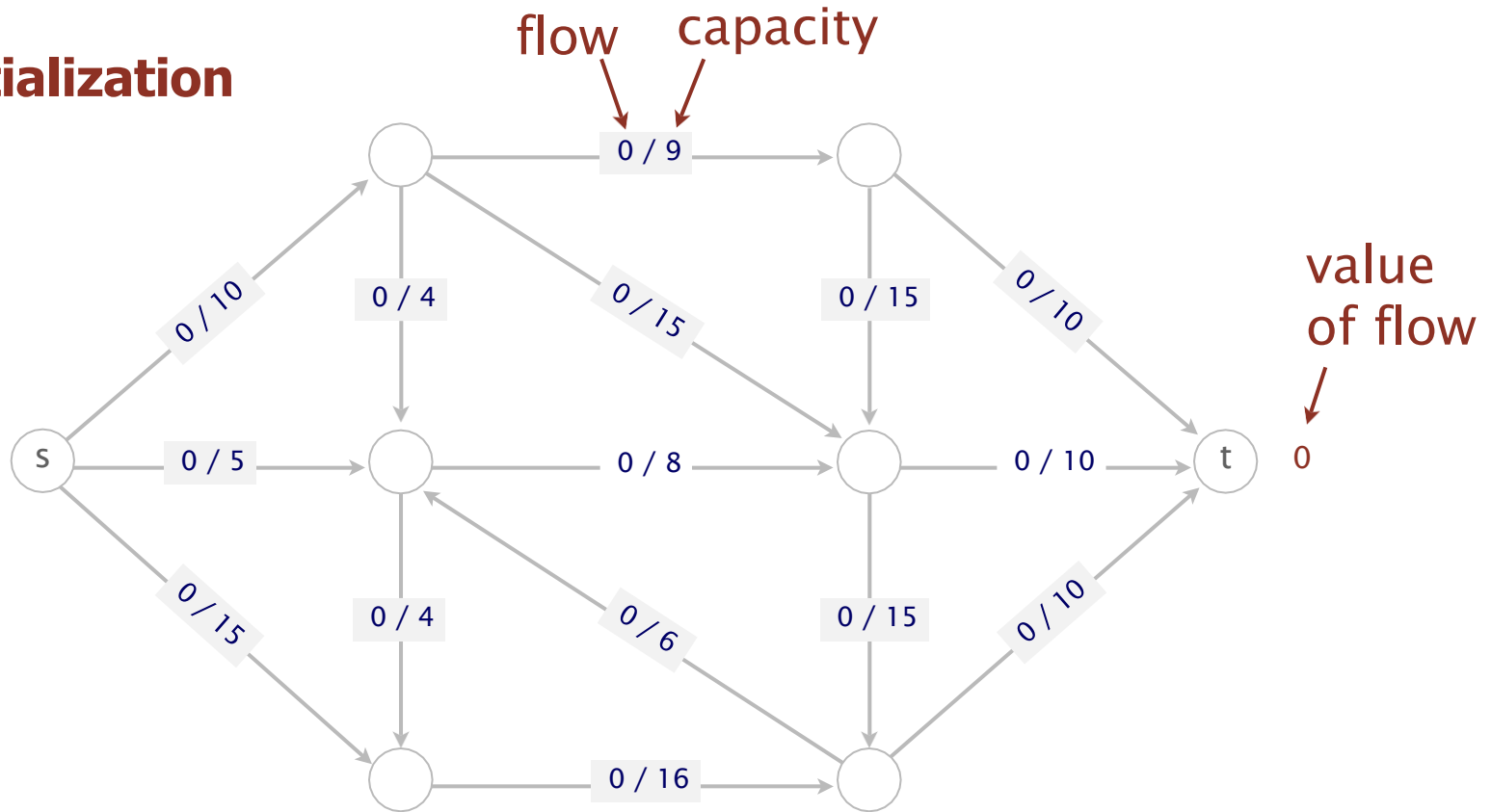
按照这个原则，d既不能超过每条向前边(v,w)的 $cap(v,w) - flow(v,w)$ ，也不能超过每条向后边(v,w)的 $flow(v,w)$ 。因此**d应该等于向前边的 $cap(v,w) - flow(v,w)$ 与向后边的 $flow(v,w)$ 的最小值**。

该方法常称作**Ford Fulkerson方法**，提供了一种解决最大流问题的思想。

Ford-Fulkerson方法

初始化: Start with 0 flow.

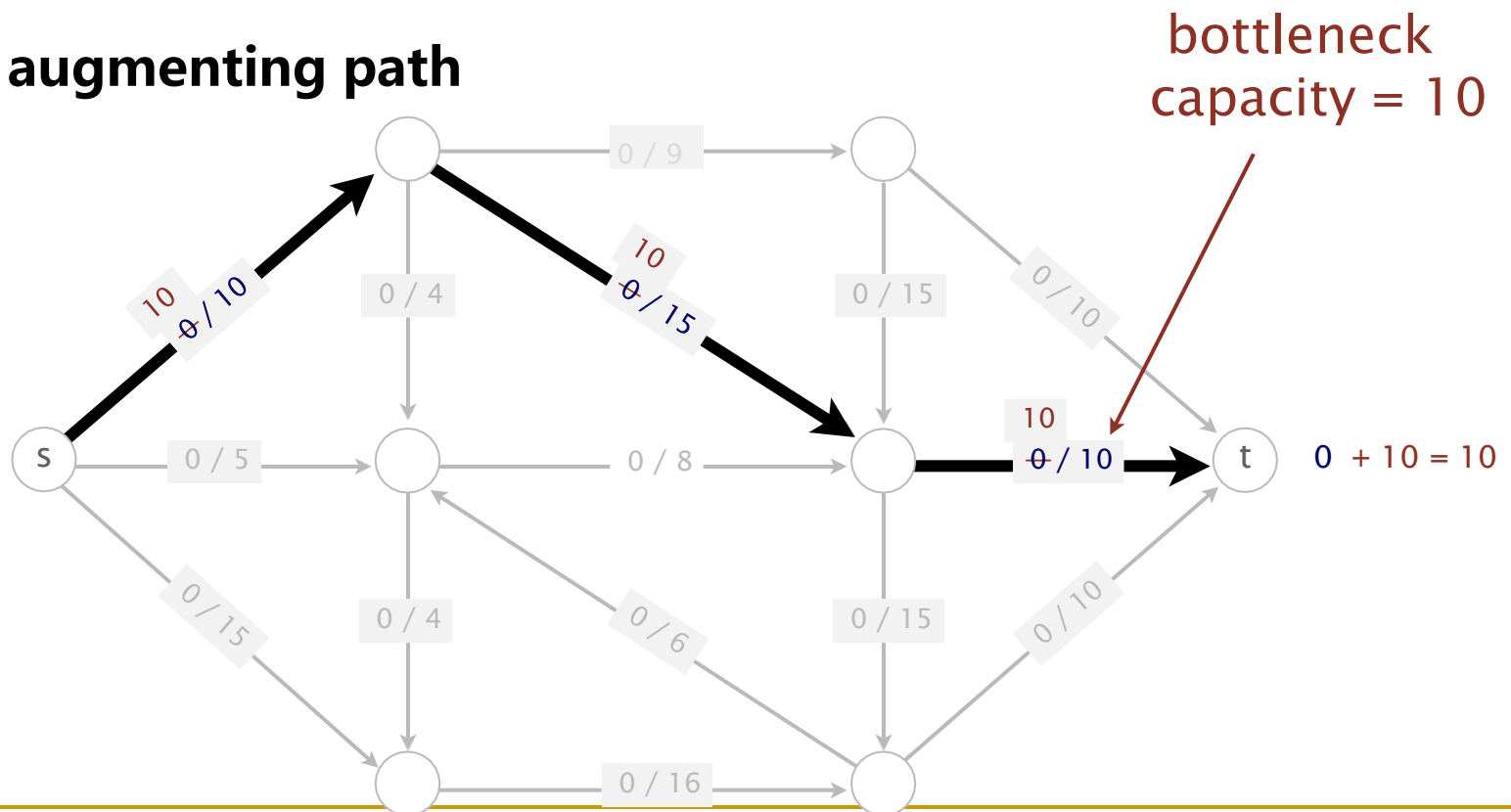
initialization



基本思想：沿着增广路增加流量

可增广路径：在残流网络中找到一条从 s 到 t 的可增广路径，满足：
增广路径上的每条边的剩余容量大于0

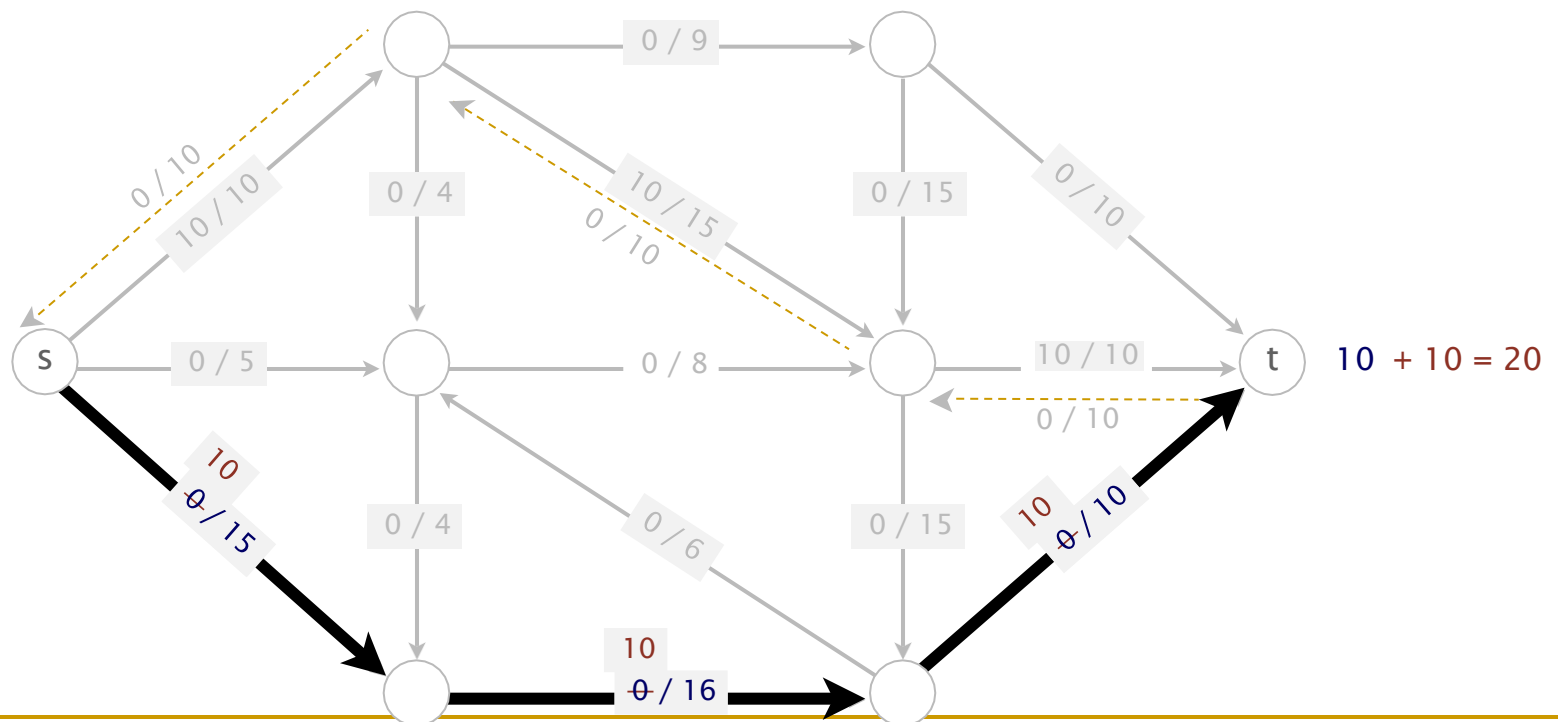
1st augmenting path



基本思想：沿着增广路增加流量

可增广路径：在残流网络中找到一条从 s 到 t 的可增广路径，满足：
增广路径上的每条边的剩余容量大于0

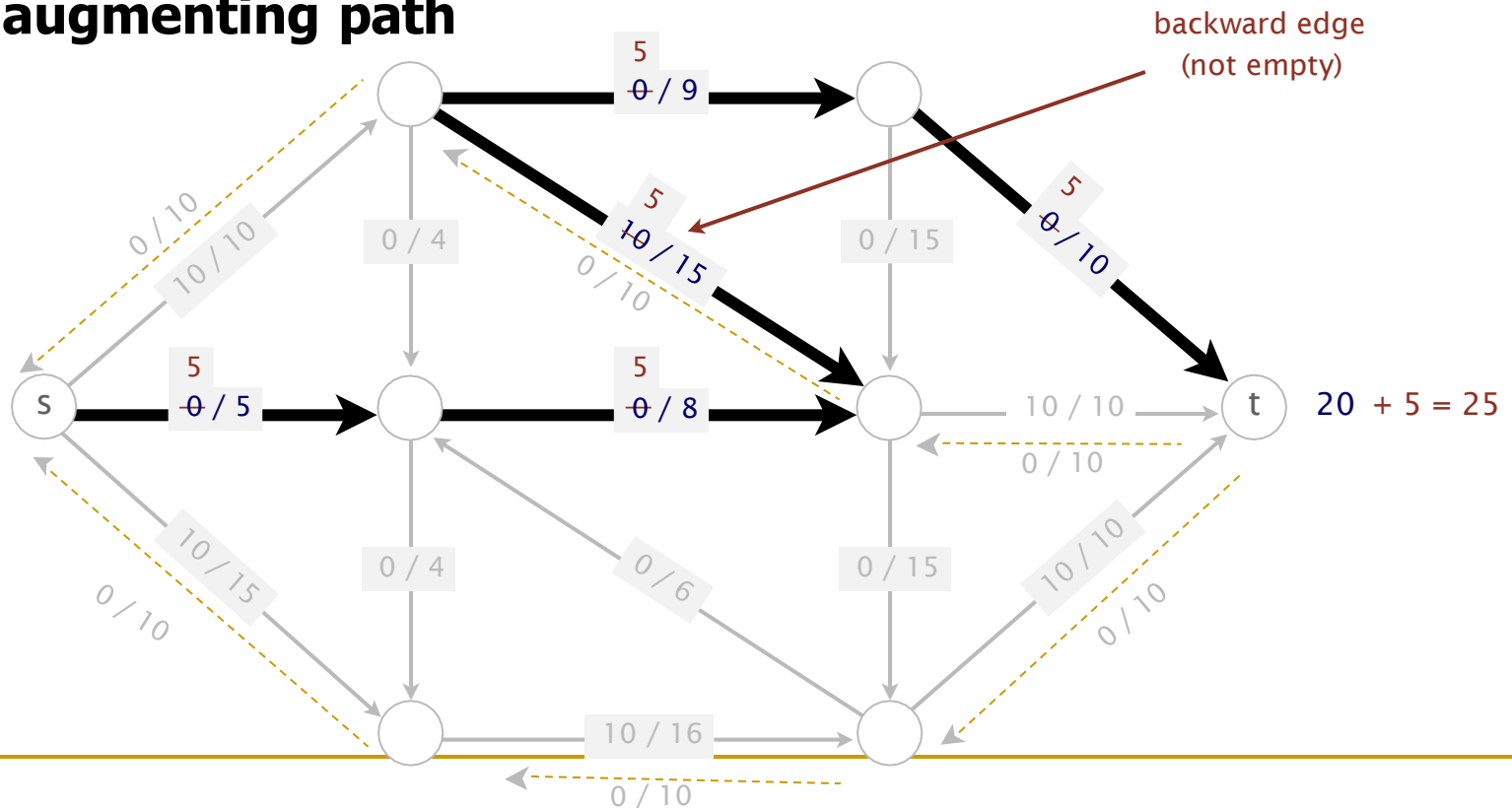
2st augmenting path



基本思想：沿着增广路增加流量

可增广路径：在残流网络中找到一条从 s 到 t 的可增广路径，满足：
增广路径上的每条边的剩余容量大于0

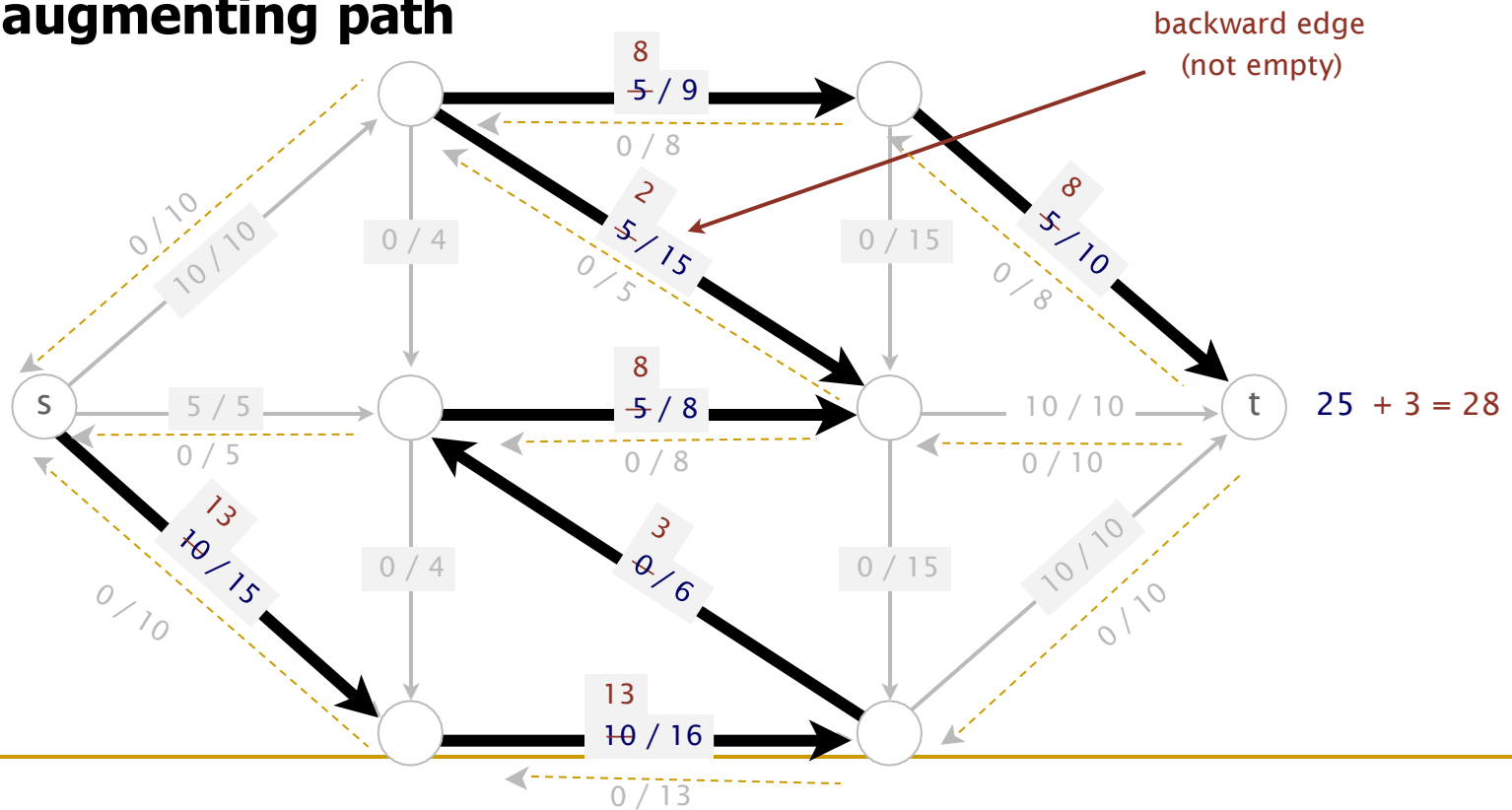
3rd augmenting path



基本思想：沿着增广路增加流量

可增广路径：在残流网络中找到一条从 s 到 t 的可增广路径，满足：
增广路径上的每条边的剩余容量大于0

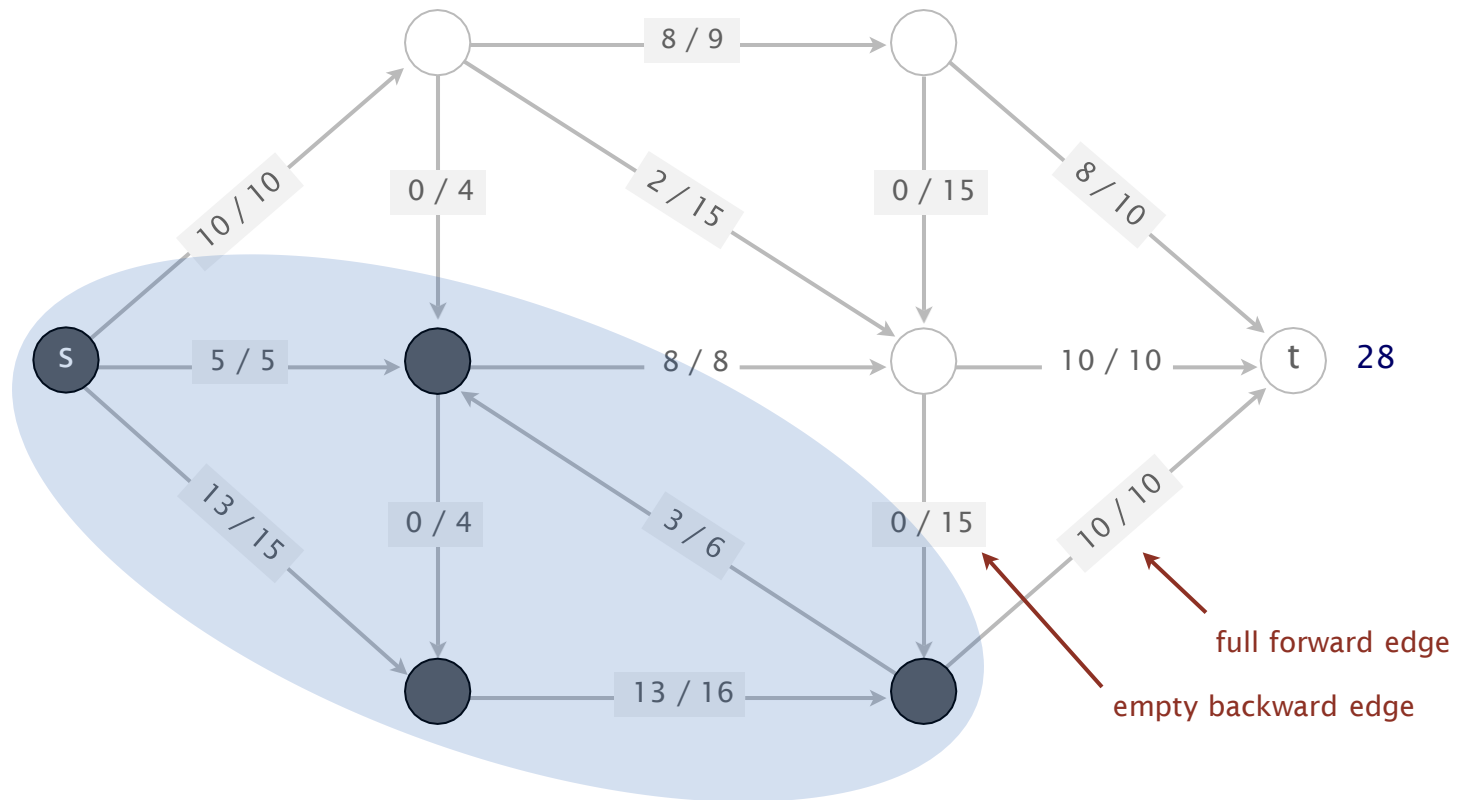
4th augmenting path



基本思想：沿着增广路增加流量

当无法找到增广路径时，算法终止

no more augmenting paths



最大网络流增广路算法描述

Ford-Fulkerson方法基于寻找增广路径的思想，以零流作为可行流，通过不断寻找增广路径来增加流量，直到无法找到增广路径为止。

该算法的步骤如下：

1. 创建一个残流网络，并初始化所有边的流量为0。
2. 当存在从源节点到汇节点的增广路径时：
 - 2.1 使用**DFS/BFS**寻找一条可增广路径。增广路径上的每条边的剩余容量大于0。
 - 2.2 沿此可增广路径增流：通过更新路径上边的剩余容量来增加流量，**即减少正向边的剩余容量，增加反向边的剩余容量。**
3. 当无法找到增广路径时，算法终止。此时，得到的流就是最大流。

最大网络流增广路算法的广义的算法

```
MAXFLOW(const Graph &G, int s, int t, int &maxflow) :  
    G(G), s(s), t(t), st(G.V()), wt(G.V()), maxf(0)  
{ while (DFS()) //dfs找到网络中的一条从s到t的可增广路, 存储在向量st中  
    augment(s, t);  
    maxflow+=maxf;  
}  
  
void augment(int s, int t) //沿可增广路计算可增广量d, 然后沿可增广路增流  
{ int d = st[t]->capRto(t); //capRto() 残流网络中的边容量  
  for (int v = ST(t); v != s; v = ST(v)) //函数ST(v)返回增广路中结点v的父结点  
    if (st[v]->capRto(v) < d) d = st[v]->capRto(v);  
  maxf+=d;  
  st[t]->addflowRto(t, d); //改变残流网络中边的流量  
  for ( v = ST(t); v != s; v = ST(v))  
    st[v]->addflowRto(v, d);  
}
```

算法效率由2个因素所确定:

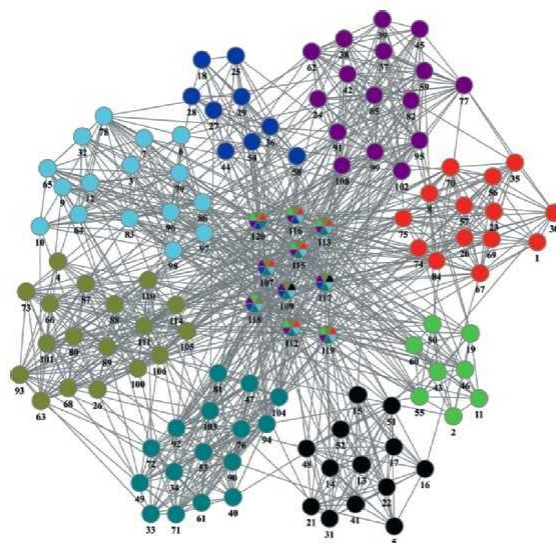
- (1) 整个算法找增广路的次数;
- (2) 每次找增广路所需的时间。

最大网络流增广路算法的具体实现

- 最初的Ford-Fulkerson算法采用DFS/BFS
- Edmonds-Karp算法(**最短路增广路算法**)
 - 优先级优先搜索(PSF): 残存网络中选择的增广路径是一条从源结点s到汇点t的**最短路**
- **最大容量增广路算法**
 - 优先级优先搜索(PSF): 残存网络中选择的增广路径是一条从源结点s到汇点t的**最大容量的路径**

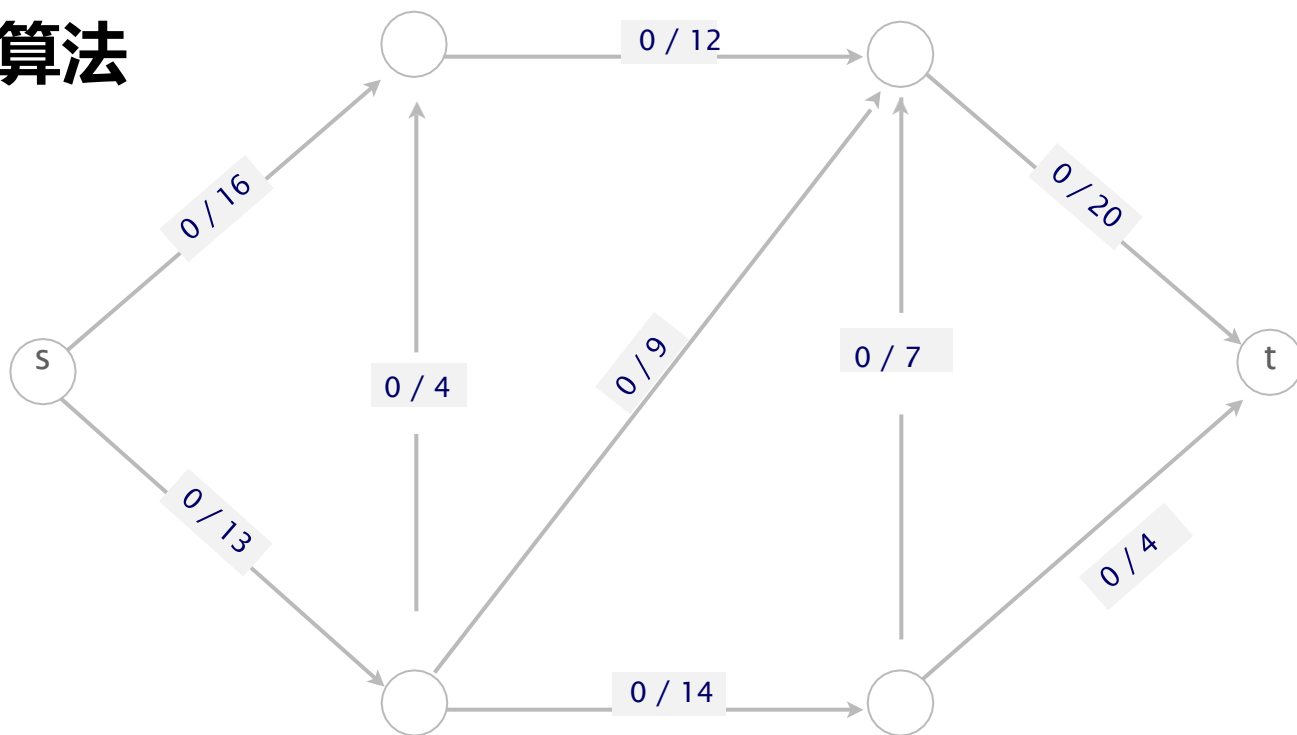
最大网络流的应用

- 交通流量优化
- 计算机网络传输优化
- 电力系统优化电力分配
- 医疗资源调度，确保医疗资源的最大利用
- 社交网络，最大限度地实现信息流传递
-



练习题

- Ford-Fulkerson算法采用DFS/BFS
- **最短路增广路算法**
- **最大容量增广路算法**



习题：

- (1) 教材 算法分析题8-2单源最短路问题表示为线性规划问题
- (2) 教材 算法分析题8-6用单纯性算法求解线性规划问题
- 在canvas上提交